CECO_DEMO · 마스터 가이드 · 2026-10-07 코드 기준

베어링이 닳던 날

공장의 값 하나가 AI까지 올라갔다가, 사람의 승인을 받아 다시 설비로 내려와 수리가 되기까지 — 구조와 데이터의 길

이 문서를 읽는 법

이 문서는 두 가지로 되어 있습니다. 시스템 전체를 빠짐없이 담은 **그림 한 장**, 그리고 그 그림 위를 처음부터 끝까지 지나가는 **이야기 하나**입니다. 숫자보다 구조를 봅니다. 어떤 부품이 무슨 일을 하는지, 누가 누구에게 어떤 프로토콜로 말하는지, 데이터 하나가 어디서 태어나 어떻게 이름과 모양을 바꾸고 어디에 저장되며 누가 다음에 읽는지를 따라갑니다.

이야기 속 동그라미 숫자(예: (18))는 그림의 화살표 번호입니다. 숫자를 따라가면 지금 이야기가 그림의 어디를 지나는지 알 수 있고, 이야기를 끝까지 읽으면 그림의 화살표를 하나도 빠짐없이 지나가게 됩니다. 괄호 안에는 그 구간의 프로토콜 · 제품 · 토픽을 적었습니다.

이 공장은 훈련용입니다. 진짜 기계 대신 컴퓨터가 물리 법칙으로 온도 · 압력 · 전류를 계산하고, 설비 시간은 실제보다 훨씬 빠르게 흐릅니다. PLC부터 AI까지 나머지는 실제로 돌아가는 제품과 프로토콜 그대로입니다.

이야기는 이렇게 흐릅니다. 서막에서 시스템이 깨어나고, 1~5장에서 설비의 값이 PLC · 공장 게시판 · 담장 · 사무실을 지나 위로 올라갑니다. 6~7장에서 고장이 나고 AI가 원인을 가리며, 8장에서 사람이 승인하고, 9~10장에서 승인된 일이 세 겹의 검사를 거쳐 설비로 내려가 수리와 회복 확인까지 끝납니다. 11장은 같은 길을 쓰는 다른 손님들입니다. 그다음 일이 순조롭지 않을 때의 갈림길, 데이터 리니지 요약, 알려진 한계가 이어집니다.

1. 시스템 아키텍처 한 장
2. 서막 · 설계도 한 장, 그리고 깨어나는 순서
3. 1장 · 이 공장, 그리고 교반기
4. 2장 · 기계 옆의 조종사, PLC
5. 3장 · 통역사와 공장 게시판
6. 4장 · 담장을 넘다: 구역 라우터와 DMZ
7. 5장 · 사무실의 장부와 실시간 검사관
8. 6장 · 베어링이 닳기 시작한다
9. 7장 · AI가 원인을 가린다
10. 8장 · 사람이 결정한다
11. 9장 · 다시 설비로: 세 겹의 검사
12. 10장 · 돌아오는 대답, 회복, 보고
13. 11장 · 같은 시간, 다른 손님들
14. 갈림길 · 순조롭지 않을 때
15. 리니지 · 데이터 리니지 요약
16. 한계 · 알려진 한계 (2026-10-07 코드 기준)
17. 저장소 밖 사실의 출처

시스템 아키텍처 한 장

띠 하나가 층 하나입니다(위 L5 AI · 업무 → 아래 L0 현장 설비). 카드는 하는 일 단위이고, 카드 아래 회색 꼬리표가 실제 컨테이너 · 포트, 카드 마지막 줄이 볼륨입니다. 화살표 139개에는 번호와 프로토콜 · 토픽 · 표 이름을 붙였고, 같은 번호가 뒤의 이야기에 나옵니다. 구성요소 69개 · compose 서비스 37개 · 볼륨 26개 · 망 3개 · Kafka 토픽 10개가 모두 이 그림에 이름으로 들어 있습니다(빌드가 대조).

데이터(상향) · 경보 · 분석 · 조치 명령(하향) · 승인 · 업무 · 조회 · 호출 · 감시 · 기동 · 1회 · 사람

L5
AI · 화면 · 분석 (5_ai)
▸ 사건 조회 · 원인 분석 · 정비 판단 · 지식 게시

담당자 · 운전원 · 강사 · 시험 도구
같은 PC 브라우저 · 사무실 망 안 시험 도구
AI 화면 · FUXA · 현장 패널 · 강사 화면
엣지 · PLC 편집기 · OT MQTT 공개 포트는 모두 127.0.0.1

74

단일 포털 (Vue 3)
nginx 1.28 · 업무와 사람 조작 분리
한 화면: 설비 상태 · 자동 이상 대응
폴링 · 실행 기록 스트림(SSE)
전체 흐름도 항상 표시 · 검토 의견 · 승인
FUXA: 포털 안 운전원 제어 패널
web :8080 → 38180

75

사건 조회 · 자동 분석 선별
knowledge 백그라운드
사건 · 이벤트 · 근거 조회
조건 맞는 사건만 분석 시작
knowledge :8000

78

원인 분석 · 검토 (LangGraph)
조사 → 사람 검토(멈춤) → 실행
분석 도구 · 구조화 출력(인용 필수)
검토 대기 지점을 DB에 저장
knowledge

80

고장 온톨로지 · 정비 판단 엔진
결정론 계산(LLM 아님)
증상 → 고장 모드 추적 · 매뉴얼 벡터 검색
자격 · 규칙 제외 · 손익 순위 · 뒤집힘
knowledge

98

지식 스튜디오
업로드 · 구조 추출 · 의미 후보 · 검토 게시
YAML · Markdown · TXT만
검토 묶음을 한 트랜잭션으로 병합 게시
후보 생성 기록은 PostgreSQL ai
볼륨 knowledge-data · knowledge-uploads · knowledge-output
knowledge

OpenAI (밖)
api.openai.com 직접 호출
gpt-6-luna · Responses API · 재시도 없음
임베딩 text-embedding-3-small

76 분석 · 승인 / 반려
89 승인 → 재검사
116 지식 스튜디오
79 HTTPS Responses
85 임베딩
97 검토 게시
81 근거 도구
120 재검사: 판단 재계산
84 사실값 읽기
117 후보 생성 기록
135 사람의 이상 발생 · 초기화
118 공정 대시보드 · 파이프라인
137 진행 중 정비 확인
77 사건 조회 · 후보 선별
82 분석 실행 · 체크포인트
83 Bolt · 벡터 검색

L5
AI · 실행 · 업무 판정 (5_ai)
▸ 근거 · 승인 재검사 · 정비 계획 · 사건 접수 · 응답 판정

사람용 설비 조작 접점
HTTP · 포털 전용 인증
이상 발생 6종 · 초기화 · 운전원 자동 연결
호스트 공개 접점으로만 설비에 연결
진행 중인 정비 확인 · AI에는 계정 없음
portal-controls :8090

121

설비 근거 · 파이프라인 상태
근거 조회
DMZ 사본 InfluxQL(읽기 전용)
Neo4j 설비 · 문서 · 상하한
화면용 파이프라인 상태(Flink 잡 수)
knowledge

냉각 조치 후 온도 관측
관측만 · 명령 없음
TT-101이 목표 근처에 머무는지 판정
냉각 대응안이 지금 안 나와 쉬는 중
knowledge

승인 재검사 · 실행 넘김
대응안 처리
기한 · 사건 버전 · 설비 지문 · 지식 지문
정비 계획은 판단 엔진 다시 계산 → 실행기
한 건 조치(정지 · 냉각) 길은 지금 안 쓰임
knowledge

92

정비 계획 실행기
상태 기계
단계별 작업 요청 → 결과 판정
회복 판정 → 보고서 → CMMS 작업 이력
knowledge

업무 서비스: AI 사건 접수
business 컨테이너 접수 스레드
sensor.alerts 처음부터 → 검증 · 원본 보관
상관 묶기 · 저장 뒤 commit
business

업무 서비스: 응답 기록 · 시간 판정
business 판정 루프
ACK · 만료 · 운전원 대기 · 정비 · 재관측
명령 불일치면 COMMAND_DISAGREE 경보
business

119 재검사: 근거 · 공정
96 관측 기록
95 회복 판정
91 request.approved (지금 안 쓰임)
93 단계 요청 · 작업 이력
100 임베딩
87 Bolt 읽기
73 AI DB 받은 원본 · 사건
70 요청 사건 · 감사 (ops)
72 sensor.alerts
128 태그 · 작업 정의
90 요청 · 감사 (지금 안 쓰임)
94 request.approved
133 관계 그래프 · 누락 조회

L4
IT · 업무 정본 · 지식 지도 (4_it · 5_ai · shared)
▸ 등록부 · 한 번 쓰면 고치지 않는 정본 · 온톨로지

설비 등록부 (정본)
등록부 파일 하나 → 생성기
설비 · 신호 · 명령 · 작업 정의 · 요청자 · 승인자
엣지 · PLC · 화면 · 허용 목록 · 규칙 범위 · DB 표를 만듦

126

업무 DB 정본 (PostgreSQL 18.6)
DB plant · DB ai · 추가 전용 표 · 계정 분리
workflow · alert · audit · enterprise · registry
ai: 사건 · 분석 · 대응안 · 체크포인트 · 지식 후보
볼륨 pg-data
postgres :5432

지식 지도 Neo4j
5.26 Community · 벡터 색인
신호 → 증상 → 고장 모드 → 원인 · 점검
결정 · 대안 · 규칙 · KPI · 매뉴얼 절
볼륨 graph-data · graph-logs
graph :7687

99

그래프 채우기
묶음 A · B · C + 임베딩
성공해야 knowledge 기동
키 없으면 임베딩 없이 성공
graph-seed (1회)

129 설비 · 센서 노드
124 FUXA 화면 설정
60 요청 · 승인 · 감사
88 Flink 잡 상태(화면용)

L4
IT · 감시 · 이력 (감시 프로필)
▸ 시스템 건강 · 결과 이력 그래프(기본 기동에는 없음)

127 규격 상한
71 audit.copy · events · DISAGREE

IT 이력 적재기
Bento 1.21.2 · 감시 프로필
clean · score · alerts → 라인 프로토콜
it-history

63

IT 결과 기록장
InfluxDB 2.9.1 · 감시 프로필
process · anomaly · alerts
볼륨 it-influx-data · it-influx-config
it-influx :8086

64

Grafana
13.2.3 · 익명 Viewer · 감시 프로필
대시보드: 공정 · 이상 점수 · 경보 · 인프라
DMZ 원천은 연결만(쓰는 패널 없음)
볼륨 grafana-data
grafana :3000

111

IT Prometheus
3.13.4 · 감시 프로필
IT 긁기 + DMZ federate
규칙 경보 → Alertmanager
볼륨 prometheus-data
prometheus :9090

67

kafka-exporter · cAdvisor
1.10.0 · 0.60.6 · 감시 프로필
소비 랙 · 컨테이너 자원
cAdvisor는 특권 실행
:9308 · cadvisor

102 현장 패널 (라우터)
139 운전원 자동 로그인
122 엣지 흐름 · 태그
130 OT MQTT (라우터)
114 기록장 지표
68 인프라 경보
134 태그 → 설비 지도
113 수집기 지표
62 clean · score · alerts
112 Flink 지표
69 plant.status · responses

L4
IT · 실시간 분석 · 경보 묶기 (4_it)
▸ Flink 검사 · 모델 · Alertmanager

115 소비 랙 조회
131 엣지 편집기 (라우터)

잡 제출기
제출 스크립트
SQL 잡 · ONNX 잡 제출
모두 돌고 있으면 건너뜀
flink-job-submitter (1회)

44

Flink SQL 잡 3개
Flink 2.2.1 · 체크포인트(내부 상태)
규격: 상하한 넘으면 → CRITICAL
Z-Score: 최근 값 대비 급변 반복 → WARNING
CEP: 교반기 전류 상한 뒤 진동 상한
출력은 최소 1회
flink-jobmanager · taskmanager

Flink ONNX 잡 (오토인코더)
Java DataStream · 출력 최소 1회
중복 제거 · 결측 보간 → clean
센서 전체 창 → 재구성 오차
임계 초과 → ML 경보
flink-taskmanager

43

모델 학습기
PyTorch → ONNX
가상설비 물리로 정상 데이터 합성
이상 기준을 모델과 함께 저장
model-trainer (1회)

ZooKeeper
3.9.5 · Flink HA 메타데이터
잡 목록 · 최신 체크포인트 위치
볼륨 zookeeper-data · zookeeper-datalog
zookeeper :2181

Alertmanager (경보 묶기 · 억제)
v0.34.1 · 기본 기동
공정: asset · rule · alertname 으로 묶음
정비 중 · PLC 정지 → 라인 전체 공정 경보 누름
설비 운전 꺼짐 → 그 설비 경보만 누름
인프라 경보: 보낼 곳 없음(화면에서만)
볼륨 alertmanager-data
alertmanager :9093

56 요청 사건 · 감사
41 sensor.alerts
42 clean · score · ML 경보
45 HA 메타데이터
101 FUXA (라우터)
66 federate (라우터)
39 telemetry.raw
40 telemetry.raw

L4
IT · 번호 매긴 장부 (4_it)
▸ 토픽 · 통로이자 사본

86 InfluxQL 읽기 (라우터)

토픽 만들기
토픽 생성 스크립트
보존 · 압축 설정
있으면 건너뜀
kafka-init (1회)

38

번호 매긴 장부 Kafka 4.3.1 (KRaft 단일 노드)
단일 노드 · 자동 생성 꺼짐 · 정본은 PostgreSQL, 장부는 통로 · 사본
sensor.telemetry.raw · sensor.telemetry.clean · sensor.anomaly.score · sensor.alerts · plant.status · alerts.display — 짧게 보관
request.approved · request.events · request.responses · audit.copy — 길게 보관
볼륨 kafka-data
kafka :9092

65 원천 연결만
53 alerts.display
37 raw · plant.status · responses
59 HTTP 생산 주문
48 plant.status
51 경보 묶음 · 사건
47 HTTP 경보 API
49 억제 상태
58 request.events · audit.copy
50 webhook
61 request.approved
55 request.approved
46 sensor.alerts
52 alerts.display
132 PLC 편집기 (라우터)

L4
IT · 수집기 스트림 (4_it)
▸ DMZ ↔ Kafka ↔ Alertmanager ↔ DB 를 잇는 Bento 스트림 7개

① DMZ → Kafka
it-collector 스트림
MQTT 구독(clean session) → 토픽 3갈래
GOOD 계측만 raw
메모리 버퍼
it-collector :4195

③ 표시 경보 → DMZ
it-collector 스트림
alerts.display → AR-100/alert/display
retained · QoS 1
it-collector

④ 분석 경보 → Alertmanager
it-collector 스트림
sensor.alerts → 꼬리표 · 설명
같은 경보가 오면 끝 시각 연장
it-collector

⑤ PLC 상태 → 억제 경보
it-collector 스트림
정비 → OutOfService · 정지 → PlantStopped
운전 꺼짐 → SuppressedByDesign(설비)
it-collector

⑥ Alertmanager → 기록 · 표시
it-collector 스트림 (HTTP 수신)
ISA-18.2 묶음 ACTIVE / CLEARED · 사건
새로 뜬 공정 경보만 표시
it-collector

⑦ 승인 요청 발송
it-collector 스트림
request.approved → 정본 조회 → 게이트웨이 1회(재시도 없음) → 결과 기록
it-collector

⑧ MES 휴내 생산 지시
it-collector 스트림 (HTTP 수신)
주문 → 요청 · 승인 기록(mes-01) → 받음 응답 · request.approved
it-collector

36 MQTT 구독 (라우터)
54 AR-100/alert/display
125 허용 목록 · 스키마
57 HTTP 요청(1회)
123 PLC 상수 · 주소
136 호스트 접점 · 라우터 경유

L3
DMZ · 완충지대 (3_dmz)
▸ 공장 ↔ 사무실 우편함 · 원시 사본 · 요청 첫 겹 검사

DMZ 적재기
Bento 1.21.2 · 메모리 버퍼
tag · status · state 구독 + 브로커 $SYS → 라인 프로토콜 묶음
dmz-loader

33

DMZ 원시 사본
InfluxDB 2.9.1 · plant_raw
쓰기: 초기 관리자 토큰(적재기)
읽기: v1 it-reader (InfluxQL)
볼륨 dmz-influx-data · dmz-influx-config
dmz-influx :8086

DMZ 브로커 (우편함)
Mosquitto 2.1.2 · 계정별 ACL · 익명 금지
bridge-ot: 계측 · 상태 · ACK · 응답 쓰기 / 요청 · 표시 읽기
gateway: request/in 쓰기 · it-collector: 읽기 + alert/display 쓰기
dmz-loader · viewer · dmz-exporter
볼륨 dmz-broker-data
dmz-broker :1883

34 MQTT 5 request/in

DMZ 요청 게이트웨이 (첫 겹 ⓐ)
Node-RED 5.0.7 · 편집기 없음
토큰 · 스키마 · 허용 목록 · 요청자 · 승인자
나이 · 중복 · 발행 확인(PUBACK)
통과 → 만료 붙여 발행 → 받음
dmz-gateway :8088

35

DMZ Prometheus
3.13.4 · 감시 프로필
DMZ 긁기 + OT remote_write 수신
볼륨 dmz-prom-data
dmz-prom :9090

31 remote_write
32 MQTT 구독
108 $SYS 지표
109 적재기 지표
110 사본 지표
29 MQTT 5 브리지 out
30 브리지 in

경계
구역 라우터 (shared)
▸ 세 망을 잇는 유일한 길 · 허용 경로 외 전부 차단

138 포털 안 실시간 제어 패널

기본 차단 · 기록
iptables 기본 DROP · MASQUERADE
ESTABLISHED/RELATED만 되돌림 허용
차단 카운터 변화 기록
망 ot-net · dmz-net (둘 다 internal) · it-net
network-router

OT → DMZ 허용
DNAT · 이름은 라우터를 가리킴
MQTT → DMZ 브로커(브리지)
remote_write → DMZ Prometheus
network-router

IT → DMZ 허용
DNAT · 출발지는 라우터 주소로
MQTT 브로커 · InfluxQL 사본
federate · 요청 창구 HTTP
network-router

호스트 → OT · DMZ
127.0.0.1 → 라우터 DNAT(호스트 관문만)
FUXA · 강사 화면 · 현장 패널
OT MQTT · 엣지 편집기 · PLC 편집기
DMZ MQTT · DMZ 사본
network-router

L2
OT · 게시판 · 요청 수용 · 화면 (2_ot)
▸ 공장 게시판 · 둘째 겹 검사 · 운전원 화면

OT 허브 · 공장 게시판 (UNS)
Mosquitto 2.1.2 · 계정별 ACL · 디스크 영속
AR-100/reaction/{line}/{asset}/tag · status · ack · state · cmd
DMZ 브리지(MQTT 5): out 계측 · 응답 / in 요청 · 표시 경보
끊긴 세션의 묶은 대기열에
볼륨 ot-hub-data
ot-hub :1883

20
21

OT 작업 요청 수신기 (둘째 겹 ⓑ)
Node-RED 엣지 둘째 탭 · MQTT 5 세션
스키마 재검사 · 중복 · 허용 작업 · 범위 · 만료
PLC 가동 · 현장 통신 · 모드 · 정비 → 자동 / 대기 / 거부
운전원 대기 · 대기 목록(retain)
edge

운전원 화면 FUXA 1.3.4
SCADA/HMI · DAQ SQLite
P&ID 화면 · 알람 · 로그인(보기 · 운전 · 관리)
요청 수락 · 거부(REMOTE_MANUAL)
볼륨 fuxa-appdata · fuxa-db
fuxa :1881

25

화면 관문
Caddy 2.11.4 · basic_auth
FUXA 게스트 토큰을 바깥에서 막음
계정 없으면 화면 안 보임
hmi-gate :1882

23 AR-100/reaction/# · alert/display
24 cmd/operator · decision
103 강사 화면 (라우터)
19 cmd/operator · cmd/request
18 MQTT tag · status · ack · state
27 $SYS
26 계정 · 화면 설정
106 엣지 거부 ACK

L2
OT · 엣지 · 보조 (2_ot)
▸ PLC 읽기 · 쓰기 · 게시 · OT 감시

엣지 PLC 쓰기
같은 탭 · Modbus/TCP
운전원 명령 1차 거름(나이 · retain · 모르는 명령)
코드 · 값 먼저, 순번 마지막
실제 시각 동기
edge

엣지 수집 · 발행 탭
Node-RED 5.0.7 · edge 계정
MQTT 3.1.1
폴링 · 새 스캔 · 품질 변화 · deadband · 최대 간격
PLC 통신 상태 · 엣지 생존(LWT) · EdgeX 묶음
끊기면 디스크 큐(store-and-forward)
볼륨 edge-data(디스크 큐)
edge :1880

허브 지표 변환
Bento · 감시 프로필
$SYS → 지표
ot-hub-exporter

28

OT 감시 에이전트
Prometheus agent 모드 · 감시 프로필
OT 긁기 → DMZ로 밀어 보냄
볼륨 ot-agent-data
ot-agent

화면 넣기
계정 · 화면 설정
관리자 비밀번호 교체 · 운전원 계정
등록부에서 만든 화면 설정
fuxa-provisioner (1회)

107 엣지 지표
22 HTTP 작업 지시
16 Modbus/TCP 읽기
17 Modbus/TCP 쓰기

L1
제어 · PLC (1_control)
▸ 입력 해독 · 인터록 · 명령 검사 · 출력

Modbus 슬레이브 (엣지 쪽)
OpenPLC v4.2.4 · :502
엣지가 읽는 계측 사본 · 상태 칸
엣지가 쓰는 시각 · 명령 칸
plc

15

명령 두 채널 검사 (마지막 겹 ⓒ)
주기 안의 명령 검사 단계
운전원 채널 · 외부 요청 채널
코드 · 범위 · 시각 · 만료 · 중복 · 모드 · 비상정지 · 인터록 · 물리 → 응답 코드
plc

12

스케줄 · 출력 · 상태
스케줄 · 출력 · 상태 단계 · RETAIN(정비 모드)
생산 설정값 · 구동기 출력
되읽기 감시 · 상태 · 하트비트
볼륨 plc-data
plc

11

고압 인터록
트립 기억
PT-101 트립 기준 이상 → 펌프 정지
해제 기준 아래 + 사람 리셋 때만 해제
plc

9

입력 해독 · 현장 스위치
입력 · 감시 단계
스캔 순번 · 시각 동기 정지 감시
비상정지 · 모드 · 정비 · 현장 버튼
plc

8

Modbus 마스터 (설비 쪽)
원격 장치 설정
계측 · 패널 읽기(오류 시 마지막 값)
구동기 · 설정값 쓰기
plc

런타임 기동 · 적재기
편집기 v4.3.2로 빌드 · REST :8443
지문이 다르면 재적재 · RUN
plc

10 모드 · 정비
13 출력 쓰기
7 Modbus/TCP 쓰기
14 상태 · ACK 칸
6 Modbus/TCP 읽기

L0
현장 설비 (0_plant)
▸ 물리 계산 · 고장 · 현장 패널 · 정비팀

가상 정비팀
HTTP :8082 · 계정(접수 담당만)
안전 확인 → 격리 → 대기 → 효과 → 해제 · 소견
plant-sim

5

강사 화면 · API
HTTP :8080 · 계정
고장 주입 · 해제 · 진값(정답)
plant-sim

3

고장 · 열화 엔진
설비 시계 · 실제 시계
고장 만료 · 열화 진행 · 노이즈 · 결측
plant-sim

반응기 물리 모델
질량 · 에너지 · 압력 수지
주기 스캔 · 배속 적분
plant-sim

1
2

Modbus 데이터 저장소
pymodbus TCP :502 · 유닛 1
코일 · 레지스터 · 접속자는 PLC 하나
plant-sim

4

현장 패널
HTTP :8081 · 계정
LOCAL/REMOTE · 현장 기동정지 · 정비 켜기/해제 키
비상정지(설비가 직접 읽음) · 리셋
plant-sim

104 격리 · 작업 · 효과
105 정비 해제 키

서막 설계도 한 장, 그리고 깨어나는 순서

이 시스템에는 정본 설계도가 하나 있습니다. **설비 등록부**입니다. 설비와 제어기, 신호, 명령, 바깥에서 요청할 수 있는 작업(작업 정의), 요청을 보낼 수 있는 시스템(AI 업무 도우미 ai-ops, 생산계획 흉내 mes-01)과 승인할 수 있는 사람의 명단이 여기에 적혀 있습니다. 이름은 국제 표준 ISA-95의 설비 계층을 따라 사이트 AR-100, 구역 reaction, 라인 reactor-line-01, 그리고 설비 순으로 붙입니다. 이 계층이 그대로 공장 게시판의 칸 이름(토픽)이 됩니다.

생성기 하나가 이 등록부에서 각 망이 쓸 설정을 만들어 저장소에 함께 둡니다. 엣지의 수집 흐름과 신호 목록 (122), PLC 프로그램의 상수와 원격 장치 설정 (123), 운전실 화면 구성 (124), 접수처 요청 창구의 허용 목록과 요청서 서식(JSON Schema) (125), 업무 DB의 등록부 표 (126), 실시간 검사관의 규격 상하한 표 (127), 경보에 설비 이름을 붙여 주는 태그 → 설비 지도 (134), 업무 서비스와 AI가 읽는 태그 · 작업 정의 (128), 지식 지도에 넣을 설비 · 센서 노드 (129)입니다. 생성기에는 산출물이 등록부와 같은지만 확인하는 모드도 있습니다. 그래서 「설비를 하나 더한다」는 일은 등록부를 고치고 생성기를 다시 돌리는 데서 시작합니다. 가상설비의 물리 상수와 고장 정의만은 가상설비 쪽 설정이 정본입니다.

기동은 make 명령 한 줄입니다. 시스템 자체를 감시하는 구성(OT 허브 지표 변환 · OT 감시 에이전트 · DMZ Prometheus · IT 이력 적재기 · IT 결과 기록장 · Grafana · IT Prometheus · kafka-exporter · cAdvisor)은 기본 기동에서 빠지고 「전체 기동」에서만 켜집니다. 경보 묶기를 맡는 Alertmanager는 공정 경보의 길에 있으므로 기본으로 켜집니다. 부품마다 건강 검사가 있어 앞 부품이 건강해져야 뒤 부품이 켜집니다. 공장 쪽은 가상설비 → PLC → 엣지 순서이고, 공장 게시판은 구역 라우터와 접수처 우편함이 건강해야 켜집니다. 그래서 처음 켤 때 접수처가 뜨지 못하면 공장도 뜨지 않습니다(이미 켜진 뒤에는 접수처가 끊겨도 공장은 계속 돕니다). 운전실 화면은 게시판 뒤, 화면 문지기와 화면 넣기는 운전실 화면 뒤입니다. 사무실 쪽은 Kafka → 토픽 만들기 → 수집기(라우터 · 업무 DB도 기다림), ZooKeeper → Flink 관리자 → 작업자 → 잡 제출(토픽 만들기와 모델 학습이 끝난 뒤), Neo4j → 그래프 채우기 → AI 업무 도우미(업무 DB도 기다림) → AI 화면 순서이고, 업무 서비스는 업무 DB · 토픽 · AI 업무 도우미 뒤에 켜집니다.

잠깐 일하고 끝나는 도우미들도 이 순서 안에서 움직입니다. **토픽 만들기**는 장부 Kafka에 칸(토픽)들을 보존 기간 · 압축 설정과 함께 만들고, 이미 있으면 건너뜁니다 (38). Kafka가 모르는 토픽을 스스로 만드는 기능은 꺼 두었으므로, 모든 칸은 여기서만 생깁니다.

그래프 채우기는 지식 지도 Neo4j에 세 묶음을 게시합니다(Bolt) (99). 첫째는 등록부의 설비 · 센서와 매뉴얼들, 둘째는 고장 지식 문서와 설비의 관계, 셋째는 증상 · 고장 모드 · 결정 · 대안 · 규칙 · KPI · 실행 단계 · 회사 사실값의 이름을 담은 온톨로지입니다. 매뉴얼의 절과 개체마다 「뜻으로 찾기」용 숫자 지문(벡터)을 OpenAI 임베딩으로 붙이는데, 내용이 바뀐 것만 다시 계산합니다(HTTPS) (100). API 키가 없으면 지문 없이도 「성공」으로 끝나므로, 그때는 매뉴얼을 뜻으로 찾는 검색이 동작하지 않습니다. 이 일이 성공해야 AI 업무 도우미가 켜집니다.

모델 학습기(PyTorch)는 가상설비와 똑같은 물리 모델과 생산 일정을 그대로 돌려 정상 운전 데이터를 만들고, 모든 센서가 평소 어떻게 함께 움직이는지 배운 오토인코더(줄였다 다시 펼치는 신경망)를 학습해, 이상 판단 기준과 함께 ONNX 파일로 모델 볼륨에 남깁니다 (43). 학습 끝에는 정상 데이터의 오탐과 몇몇 고장의 탐지 여부를 스스로 재어 기록합니다.

잡 제출기는 실시간 검사관 Flink가 준비되기를 기다린 뒤 SQL 규칙 잡들과 ONNX 모델 잡을 맡깁니다 [44]. 이미 모두 돌고 있으면 건너뛰고, 일부만 돌고 있으면 중복으로 넣지 않고 실패로 끝내 사람이 보게 합니다. Flink 관리자는 쓰러졌다 일어나도 이어 가도록, 잡 목록과 최신 진행 위치(체크포인트)가 어디 있는지를 ZooKeeper에 적어 둡니다 [45]. 체크포인트 본체는 Flink 볼륨에 있습니다.

화면 넣기는 운전실 화면 FUXA의 보안 기능이 켜져 있는지 확인하고, 관리자 기본 비밀번호를 바꾸고, 운전원 계정을 만들어 로그인을 확인한 뒤, 등록부에서 만든 화면 구성과 MQTT 접속 계정을 넣습니다 [26].

사람이 쓰는 명령도 make 하나로 모았습니다. 기동 · 전체 기동 · 정지 · 볼륨까지 지우는 정지 · 상태 · 로그 · 접속 주소, 등록부를 고친 뒤 다시 만들기, 모델 다시 학습, Flink 잡 상태, 탐지 시험용 고장과 정비 시나리오 주입, 고장 해제, 가상설비 상태, 계층별 자동 검사입니다. 다시 만들기와 다시 학습에는 주의할 점이 있어 「알려진 한계」에 적었습니다.

1장 이 공장, 그리고 교반기

추가 운전 조작은 같은 포털의 설비 제어 패널에서 합니다. 포털 서버가 기존 관문을 거쳐 화면과 실시간 연결을 중계하고 [138], 사람용 접점이 운전원으로 자동 로그인합니다 [139]. 아이디나 비밀번호를 직접 입력할 필요가 없습니다. 비밀번호는 서버에만 두며, 브라우저에는 일반 운전원 세션만 전달합니다. 관리자 권한으로 연결하지 않으며, 연결이 끊겼다면 패널 안에서 다시 연결할 수 있습니다.

담당자는 하나의 포털에서 설비를 보고, 이상 상황을 발생시킨 뒤 같은 화면에서 감지와 자동 분석을 확인합니다. 포털은 사람의 조작을 전용 접점(portal-controls)에 HTTP로 넘깁니다 [135]. 이 접점은 설비 계정을 브라우저에 건네지 않고, 같은 PC의 공개 접점과 라우터를 거쳐 가상설비에 전달합니다 [136]. 사람용 연결은 기존 호스트 경로를 쓰며, 사무실에서 공장으로 직접 여는 방화벽 규칙을 더하지 않습니다. 조작 전에는 업무 쪽에 정비가 진행 중인지 확인하고 [137], 진행 중인 정비를 새 이상이나 초기화로 끊지 않습니다. AI가 받는 근거는 계속 센서 기록과 매뉴얼이며, 이 사람용 접점의 설비 계정은 받지 않습니다.

원료를 통에서 꺼내 솥(반응기)에 붓고, 알맞은 온도로 데우면서 저어 반응시킨 다음, 다 된 액체를 내보내는 공장입니다. 설비와 센서는 이렇습니다. 원료탱크 TK-101(액위 LT-101), 반응기 R-101(액위 LT-102 · 온도 TT-101 · 압력 PT-101 · pH-101 · 전도도 CT-101 · 독립 압력계 PT-102), 재킷 히터 HX-101(재킷 온도 TT-102), 냉각기 HX-102(냉각수 유량 FT-103 · 공급 온도 TT-103 · 회수 온도 TT-104), 냉각수 스트레이너 ST-103(차압 PDT-103), 공급 펌프 P-101(공급 유량 FT-101 · 펌프 전류 IT-101), 배출 밸브 CV-101(배출 유량 FT-102), 교반기 M-101(전류 IT-102 · 진동 VT-101), 그리고 제어기 PLC-01. 대부분의 센서에는 규격 상한이나 하한이 있고, 공급 유량 · 배출 유량 · 전도도는 한계 없이 관측만 합니다.

물리는 이렇게 맞물려 있습니다. 원료탱크는 자동으로 보충되고, 펌프가 원료를 반응기로 보내며, 배출 밸브가 내보냅니다. 열은 재킷 히터와 반응열로 들어오고 냉각 재킷이 빼냅니다. 반응이 열을 내므로 냉각은 정상 운전에서도 늘 켜져 있고, 실제로 빼낼 수 있는 열은 냉각수 유량과 재킷의 청결도로 정해집니다. 냉각수는 이중 스트레이너(A/B 바스켓)를 지나 재킷 코일로 들어가므로, 스트레이너가 막히면 차압이 오르고 유량이 줄어 반응기가 뜨거워집니다. 반응기 압력은 온도와 액위를 따라 오르고, 기계식 안전밸브는 실제 압력에만 반응합니다. PT-102는 PT-101과 따로 단 독립 압력계여서, 지시값만 틀어진 것과 실제로 압력이 오른 것을 가려내는 근거가 됩니다. pH와 전도도는 서로 반대로 움직입니다. 그래서 값 하나하나는 정상인데 관계가 깨지는 이상이 있고, 그것은 뒤에 나오는 AI 모델만 잡아냅니다.

가상설비에는 시계가 둘 있습니다. 서서히 진행하는 고장 · 생산 일정 · 정비 작업 시간은 배속으로 흐르는 **설비 시계**를, 순간 사건(튐 · 결측 같은 고장)과 PLC의 타이머는 **실제 시계**를 씁니다. 같은 배속 값이 AI 쪽에도 따로 설정되어 있어서, 둘이 어긋나면 시간 계산이 틀어집니다. 계측값에는 정상 잡음이 섞이고, 결측은 품질 BAD가 되며, 정비 중 격리된 계기도 BAD로 나옵니다.

오늘의 주인공은 반응기 안을 쉬지 않고 젓는 **교반기 M-101**입니다. 전류 IT-102는 모터가 얼마나 힘들게 도는지, 진동 VT-101은 축이 얼마나 떨리는지 보여 줍니다. 베어링이 닳으면 먼저 모터가 힘들어져 전류가 오르고 뒤따라 진동이 커지며, 축이 틀어진 것이라면 전류는 상한 아래인데 진동만 커집니다. 겉으로 비슷한 두 고장을 「무엇이 먼저였나」로 가려내는 것이 이야기의 열쇠입니다.

강사가 넣을 수 있는 고장은 두 무리입니다. 탐지 시험용은 히터 출력 고착(비상정지 중에도 가열), 냉각 능력 상실(냉각 명령이 켜져 있어도 열이 빠지지 않음 — 명령 상태만으로는 효과를 증명할 수 없다는 사례), 온도 결측, 압력 튐, 온도 잡음 급증, pH · 전도도 미세 드리프트(각각은 규격 안이라 AI 모델만 잡음), 베어링 마모입니다. 정비 시나리오용 열화는 증상 셋에 원인이 둘씩입니다. 온도가 오르면 스트레이너 막힘(유량↓ · 차압↑)인지 재킷 스케일(유량 · 차압 정상, 회수 온도차↓)인지, 진동이 오르면 베어링 마모(전류가 먼저)인지 축 정렬 불량(진동만)인지, 압력이 오르면 PT-101 드리프트(PT-101만 높음)인지 배출 밸브 고착(PT-102와 액위도 높음)인지. 비슷한 두 원인을 가려낼 신호가 늘 하나씩 있습니다. 열화는 오랜 기본 기한이 지나기 전에는 원인에 맞는 현장 정비만 되돌립니다. 강사의 「전체 초기화」는 정비가 아니라 고장 · 열화 · 격리를 모두 지운 새 설비로 되돌리는 것이고, 고장 이름(정답)은 사람용 이상 발생 접점에 있고 AI의 근거 조회에는 가지 않습니다.

설비 쪽 컴퓨터(가상설비, Python · pymodbus · aiohttp)는 일정한 주기로 물리를 계산하고, 결과를 번호 붙은 사물함(Modbus 데이터 저장소)에 적어 둡니다 ①. 계측값과 함께 몇 번째 스캔인지(스캔 순번)와 설비 시각도 적습니다. 이것이 모든 센서 값의 **출생지**입니다. 계산을 시작할 때마다 같은 사물함에서 구동기(펌프 · 교반기 · 히터 · 냉각기 켜짐/꺼짐)와 설정값(펌프 속도 · 밸브 개도 · 온도)을 꺼내, 물리적으로 가능한 범위로 잘라 반영합니다 ②. 「사물함에 써졌다」, 「설비가 그 값을 따랐다」, 「원하는 효과가 났다」는 서로 다른 이야기입니다. 사물함은 Modbus/TCP로 열려 있고 접속하는 쪽은 PLC 하나입니다.

현장 패널도 있습니다. 같은 PC에서 담장(라우터)을 거쳐 들어가(HTTP, 패널 계정) 102 버튼을 누르면 그 값이 사물함의 패널 칸에 적힙니다 ④. 패널에는 LOCAL/REMOTE 선택 스위치, 현장 기동 · 정지, 정비 켜기(담당 번호), 정비 해제 키, 눌린 채 걸리는 비상정지, 리셋(비상정지 해제와 인터록 리셋)이 있습니다. 순간 버튼은 누를 때마다 카운터를 하나씩 올리는 방식이라, PLC는 카운터가 바뀐 것을 보고 눌렸다고 압니다. 패널 계정은 사람이 쓰고, 이상 발생 계정은 사람용 포털 접점도 씁니다. 정비팀 계정은 공장 접수 담당만 씁니다.

2장 기계 옆의 조종사, PLC

센서 바로 옆에는 PLC(OpenPLC Runtime, IEC 61131-3 ST 프로그램)가 있습니다. PLC는 짧은 주기마다 설비의 사물함을 읽어(Modbus/TCP, PLC가 여는 쪽, 읽기 오류면 마지막 값 유지) ⑥ 숫자로 풉니다 ⑧. 한 번의 주기는 이 순서입니다. 입력 해독 → 현장 통신 · 시각 동기 감시 → 기동 초기화 → 비상정지 → 모드 스위치 → 정비 모드 → 현장 버튼 → 고압 인터록 → 두 채널 명령 검사(와 인터록 리셋) → 생산 스케줄 → 출력 → 되읽기 감시 → 상태 기록.

통신 감시는 두 가지입니다. 설비의 스캔 순번이 잠시 동안 바뀌지 않으면 「현장 통신 끊김」 입니다. 그리고 엣지가 꾸준히 PLC에 실제 시각을 써 주는데, 그 시각이 멈추면 「시각 동기 이상」 이 되어 바깥 요청을 「시각 동기 없음」 으로 거절합니다. 요청의 만료를 따질 기준 시각이 없기 때문입니다. 설비 시각은 생산 스케줄에만 씁니다.

생산 스케줄은 설비 시간으로 하루마다 새 목표(펌프 속도 · 밸브 개도 · 목표 온도)를 정하고 설정값이 천천히 따라가게 합니다. 운전원이 설정값을 바꾸면 그 설정값은 한동안 스케줄에서 빠집니다. 설비나 PLC에 「배치」 라는 개념은 없습니다. 출력은 다음 주기에 사물함에 쓰여(Modbus/TCP, 코일과 레지스터) (13) (7) 기계를 움직입니다. 되읽기 감시도 있습니다. 펌프나 교반기에 기동 출력을 냈는데 전류가 오르지 않으면 「실제로는 안 돌고 있다」 고 보고 명령을 끄고 고장 표시를 남깁니다.

PLC는 안전 브레이크이기도 합니다. 반응기 압력계 PT-101의 지시값을 매 주기 보다가 (9) 트립 기준을 넘으면 그 즉시 펌프를 끊고 트립을 기억합니다 (11). 독립 압력계 PT-102는 이 판단에 쓰지 않고, PT-101 값이 결측이면 판단에서 뺍니다. 누구에게도 묻지 않습니다. 사무실의 AI는 근거를 모아 계획을 내기까지 시간이 걸리고, 그때 서버가 바쁘거나 선이 끊길 수도 있기 때문입니다. 압력이 내려가도 저절로 풀리지 않고, 해제 기준 아래로 내려온 것을 확인한 사람이 리셋해야 풀립니다. 리셋은 잠금만 풀 뿐 펌프는 새 기동 명령을 기다립니다. 그 위로는 기계식 안전밸브가 있습니다. 실제 큰 공정에서는 이런 안전 차단을 일반 제어와 독립된 안전 계장 시스템(SIS)으로 따로 두라고 국제 표준 IEC 61511이 요구합니다. 이 훈련 공장은 PLC 한 대가 제어와 인터록을 함께 맡습니다.

비상정지는 두 길로 막습니다. 가상설비가 패널의 비상정지를 직접 읽어 PLC와 상관없이 구동기를 끄고, PLC도 비상정지를 보면 모든 구동 명령을 끕니다. 그래서 비상정지를 풀어도 기계가 저절로 다시 돌지 않습니다. 비상정지 중에는 기동 · 설정값 변경 · 모드 변경이 거절되고 정지 명령만 받습니다.

운전 모드는 셋입니다. 현장 패널의 선택 스위치는 LOCAL과 REMOTE 둘뿐입니다 (10). LOCAL로 돌리면 현장 모드, REMOTE로 되돌리면 언제나 「원격 수동」 이 됩니다. 「원격 수동」 과 「원격 자동」 사이는 운전원의 모드 명령으로만 바꾸고, 원격에서 현장 모드로 바꾸는 명령은 없습니다. 정비 모드를 켜는 길은 셋(현장 패널, 운전실 화면의 운전원, 바깥 요청 「정비 모드 켜기」)이고, 끄는 길은 현장 패널의 정비 해제 키와 정비팀의 잠금 해제 작업뿐입니다.

모드에 따라 받는 명령이 다르고, 더 제한적인 쪽이 이깁니다. 현장 모드에서는 바깥 요청을 모두 거절하고 운전원 명령은 정비 켜기만 받습니다(현장 기동 · 정지 버튼은 현장 모드에서만 받습니다). 정비 중에는 바깥 요청을 모두 거절하고 운전원은 정지와 정비 켜기만 할 수 있습니다. 원격 자동에서는 바깥 요청을 받고, 운전원은 모드 · 정비 · 인터록 리셋만 할 수 있습니다. 원격 수동에서는 운전원 명령을 모두 받고, 바깥 요청은 운전원이 수락했다는 표시가 붙은 것만 받습니다.

명령이 들어오는 칸은 둘입니다. 운전실 화면에서 오는 **운전원 채널**과 사무실 요청이 오는 **외부 요청 채널**. PLC는 순번이 바뀐 것을 보고 새 명령인 줄 알고 (15) 이 순서로 따집니다. 아는 명령인가 → 값이 범위 안인가 → (외부 요청만) 시각 동기가 살아 있나, 만료되지 않았나, 최근에 같은 요청 지문이 없었나 → 모드와 정비 → 비상정지 → 인터록(인터록 중 펌프 기동 거절) → 물리 타당성(원료탱크가 비었으면 펌프 기동 거절). 통과한 값만 출력에 반영합니다 (12). 결과는 응답 코드 하나로 남습니다. 수용, 현장 모드, 정비 중, 모드, 범위, 인터록, 만료, 중복, 모르는 명령, 비상정지, 시각 동기 없음, 물리 불가 가운데 하나입니다. 인터록 리셋은 운전원 채널에서만 받고, 운전원 채널에는 만료 · 중복 · 시각 검사가 없습니다.

PLC는 지금의 상태를 정해진 **상태 칸**에 늘 적어 둡니다 [14]. 프로그램 실행 표시, 모드, 정비 여부와 담당 번호, 인터록, 현장 통신, 비상정지, 구동 출력, 설정값, 두 채널 각각의 마지막 응답 순번과 응답 코드(외부 요청은 요청 지문까지), 주기마다 오르는 하트비트, 되읽기 고장 표시입니다. 이것이 **PLC 상태**와 **명령 결과(ACK)**의 출생지입니다. 전원이 나가도 남는 영역(RETAIN)에는 정비 모드와 담당 번호만 저장하고, 인터록 · 모드 · 구동 명령 · 설정값 · 중복 기억은 다시 켜면 초기값으로 돌아갑니다.

PLC 프로그램은 OpenPLC Editor를 화면 없이 돌려 런타임 묶음으로 만들고, 런타임 이미지 안에서 컴파일합니다. 기동할 때 적재기가 첫 관리자 계정을 바로 만들고(첫 계정은 누구나 만들 수 있는 규칙 때문입니다), 설치된 프로그램의 지문이 다르면 묶음을 다시 올려 실행 상태로 둡니다. 편집기 REST 창구는 같은 PC에서 라우터를 거쳐서만 열 수 있습니다 (HTTPS) [132].

PLC와 설비가 쓰는 Modbus는 1979년 Modicon이 PLC용으로 발표한 프로토콜로, 인증도 암호화도 없습니다. 닿기만 하면 누구든 「펌프 켜」를 쓸 수 있습니다. 공장을 담으로 둘러싸야 하는 가장 근본적인 이유입니다.

3장 통역사와 공장 게시판

PLC는 「몇 번 칸에 숫자 얼마」라고만 말합니다. 엣지(Node-RED)라는 통역사가 짧은 주기로 PLC의 계측 칸과 상태 칸을 읽습니다(Modbus/TCP, 엣지가 여는 쪽) [16]. 계측은 스캔 순번이 바뀐 새 스캔일 때만 보고, 센서마다 품질이 바뀌었거나, 값이 기준 폭(deadband)보다 크게 변했거나, 마지막 보고 뒤 정해진 설비 시간이 지났으면 쪽지로 만듭니다. 배속이 빨라서 실제로는 거의 모든 센서가 매 스캔 보고됩니다.

이 쪽지가 **UNS 메시지**입니다. 사물함의 숫자가 여기서 처음으로 이름을 얻습니다. 칸 이름은 「공장/구역/라인/설비/종류/센서」, 곧 AR-100/reaction/reactor-line-01/M-101/tag/IT-102이고, 내용(JSON)은 값 · 품질 · 잰 시각 · 스캔 순번 · 설비 시각입니다. 품질은 셋입니다. 값이 없거나 비정상이면 BAD, PLC가 「현장 통신 끊김」을 알리고 있으면 STALE, 그 밖에는 GOOD. 엣지와 PLC 사이 연결이 끊겼을 때도 엣지는 마지막 값을 STALE로 한 번 붙이고 PLC 통신 상태 칸에 「끊김」을 남깁니다. 서식에 정의된 UNCERTAIN은 지금 만드는 곳이 없습니다. 엣지는 쪽지를 공장 게시판에 붙입니다 (MQTT 3.1.1, QoS 1) [18].

PLC 상태는 바뀐 항목만, 그리고 일정한 간격으로 전부 다시 「마지막 한 장 남기기(retain)」로 붙습니다. 늦게 구독한 쪽도 지금 상태를 바로 알 수 있게 하기 위해서입니다. 「운전 상태」는 PLC 프로그램이 돌고 있느냐(하트비트가 계속 오르느냐)를 뜻하고, 설비가 멈춰 있어도 PLC가 돌면 RUN입니다. 명령 결과(ACK)는 응답 순번이 바뀔 때 붙고, 외부 요청의 결과에는 요청 지문이 있어 어느 요청의 결과인지 되찾습니다. 엣지 자신의 생존도 「태어남」 쪽지와 끊기면 브로커가 대신 붙이는 유언(LWT)으로 알립니다. 수업용으로 같은 계측을 EdgeX 형식 묶음으로도 붙이는데, 이 칸은 공장 안에만 머뭅니다.

게시판과 끊기면 엣지는 보낼 쪽지를 디스크 큐에 모아 두었다가 다시 이어지면 순서대로 먼저 붙입니다(store-and-forward). 한도를 넘으면 새 쪽지를 버리고 버린 수를 셉니다. 엣지 편집기는 로그인이 필요하고 읽기 전용이며, 같은 PC에서 라우터를 거쳐 열 수 있습니다 [131]. 엣지는 켜질 때마다 등록부에서 만든 흐름을 다시 배포하므로, 편집기에서 고친 것은 남지 않습니다.

공장 게시판(OT 허브, Eclipse Mosquitto)은 공장 안의 모두가 연결하는 단 하나의 곳입니다. 쪽지가 붙으면 그 칸을 구독한 쪽에 즉시 밀어 보내고(push), QoS 1이면 받았다는 확인(PUBACK)이 올 때까지 다시 보냅니다. 전해 준 쪽지를 보관하지는 않습니다. 생중계 방송국과 같아서, MQTT는 보관하는 곳이 아니라 전달하는 곳입니다. 다만 세션을 유지하는 구독자가 잠깐 끊겼으면 그 몫을 대기열에 모아 두고, 그 내용을 주기적으로 디스크에 저장해 게시판이 다시 켜져도 이어집니다.

익명 접속은 막혀 있고, 계정 다섯(엣지 · 접수 담당 · 운전실 화면 · 보기 전용 · 감시)이 각자 허락된 칸(ACL)만 붙이고 읽습니다. 운전원 명령 칸과 외부 요청 명령 칸은 쓰는 계정이 다릅니다. 운전원 명령 · 요청 명령 · 대기 목록 · 운전원 결정 · EdgeX 묶음은 공장 안에만 머무는 칸입니다. 같은 PC의 사람은 라우터를 거쳐 보기 전용 계정으로 게시판을 구독해 볼 수 있습니다 (130).

같은 게시판을 운전실 화면(FUXA, SCADA/HMI)도 구독해서(MQTT) 교반기 전류가 곧바로 운전원 모니터에 뜹니다 (23). 화면은 반응기 공정도(P&ID) 한 장에 공정 흐름 · 운전원 조작 · 전체 계측 · 설정값 · 사무실 분석 경고(참고) · 받은 요청 구역이 있습니다. 알람은 FUXA가 스스로 판단합니다. 센서별 규격 한계, 고압 인터록 트립, 비상정지 눌림, 현장 장치 통신 끊김, PLC 통신 끊김. 계측은 화면 안 SQLite 기록에 잠시 남습니다. 보는 것은 손님 계정으로도 되지만, 명령을 내리려면 운전원 계정, 화면을 고치려면 관리자 계정으로 로그인해야 합니다.

화면 앞에는 계정을 묻는 문지기(Caddy, HTTP 기본 인증)가 있습니다. 이 FUXA는 토큰 없이 들어온 요청에 손님 토큰을 내주고 그것을 끌 설정이 없어서, 바깥에 한 겹을 더 둔 것입니다. 같은 PC에서 담장을 지나 들어온 사람도 (101) 문지기 계정이 없으면 화면을 볼 수 없고 (25), 들어간 뒤에는 FUXA 자체 로그인이 한 번 더 있습니다.

운전원이 버튼을 누르면 명령은 반대로 흐릅니다. 게시판의 운전원 명령 칸에 붙고(MQTT) (24), 엣지가 받아 (19) 먼저 모양을 거릅니다. 읽을 수 없는 쪽지, 모르는 명령, 남겨진(retain) 명령, 너무 오래된 명령, 정수로 바꿀 수 없는 값이면 PLC에 쓰지 않고 엣지 이름으로 거절 결과를 붙입니다 (106). 통과하면 PLC의 운전원 채널에 명령 코드와 값을 먼저 쓰고 순번을 마지막에 씁니다(Modbus/TCP) (17). PLC가 2장의 검사를 거쳐 응답 코드를 남기면, 그 결과가 같은 길로 올라와 운전실 화면의 「마지막 명령 응답」에 뜹니다. 운전원 명령은 구동기 기동과 정지, 설정값, 원격 수동 · 원격 자동 전환, 정비 켜기, 인터록 리셋입니다. 현장 모드 선택과 정비 해제는 화면에서 할 수 없습니다.

여기까지는 모두 공장 안입니다. 사무실과의 연결이 통째로 끊겨도 값 보기 · 버튼 · 알람 · 안전장치는 그대로 돕니다. 바깥 연결을 끊어 본 시험에서도 공장 안은 멈추지 않았습니다.

시스템 자체의 건강을 보는 감시 구성(평소 꺼 둠)도 공장 안에 있습니다. 게시판의 상태 칸($SYS: 연결 · 버린 쪽지 · 오간 쪽지 · 브리지 연결 · 가동 시간)을 지표로 바꾸는 변환기(Bento) (27)가 있고, 감시 에이전트(Prometheus agent 모드)가 그 지표 (28)와 엣지의 지표 (107)를 긁어 잠시 모아 두었다가 접수처로 밀어 보냅니다(4장).

4장 담장을 넘다: 구역 라우터와 DMZ

이제 전류 값이 공장 담장을 넘어야 합니다. 사무실은 인터넷 · 메일 · 외부 업체와 이어져 있어 언제든 뚫릴 수 있습니다. 사무실이 뚫렸을 때 공격자가 공장 문까지 바로 가면 안 됩니다. 그래서 공장(OT), 접수처(DMZ), 사무실(IT)은 서로 다른 망이고, 공장과 접수처 망은 바깥으로 나가는 길 자체가 없는 내부 전용 망입니다. 세 망에 모두 닿는 것은 **구역 라우터**

(iptables) 하나뿐입니다.

라우터는 기본이 「모두 막기」이고, 미리 정한 길만 목적지로 넘겨줍니다(DNAT). 이미 연 연결의 되돌아오는 길만 허용하고, 막은 횟수의 변화를 기록합니다. 각 망에서 「dmz-broker」 같은 이름은 실제 상대가 아니라 라우터를 가리키도록 해 두었고, 넘겨줄 때 출발지를 라우터 주소로 바꾸므로(MASQUERADE) 받는 쪽은 누가 보냈는지를 주소가 아니라 계정과 권한(ACL)으로 구분합니다. 열어 둔 길은 공장 → 접수처(MQTT, 감시 기록), 사무실 → 접수처(MQTT, 원본 사본 조회, 감시 기록 가져오기, 요청 창구), 그리고 같은 PC의 사람이 공장 화면들과 접수처에 들어가는 길입니다. 사람 길은 PC 쪽 관문 주소에서 온 것만 받으므로 사무실 망의 컨테이너가 이 길로 공장에 들어갈 수는 없습니다. 접수처나 사무실에서 공장으로 새로 거는 길은 없고, 시험에서도 그런 연결은 모두 막혔습니다. 사람이 여는 문은 모두 이 PC 안(127.0.0.1)에만 열려 있고, 사무실 부품(Kafka · Flink 화면 · 결과 기록장 · 업무 DB · Grafana · Prometheus · Alertmanager · Neo4j · AI 업무 도우미 · AI 화면)은 라우터 없이 PC에 바로 열려 있습니다. 산업 보안 표준 IEC 62443이 말하는 「구역(zone)과 관문(conduit)」을 작게 옮긴 구조입니다.

접수처는 다리가 아니라 다리 한가운데의 면회실입니다. 규칙은 하나, **전화는 안쪽에서만 건다**. 「다리」는 공장 게시판이 접수처 우편함(DMZ 브로커, Mosquitto)에 먼저 걸어 둔 전화 한 통, MQTT 브리지입니다(MQTT 5, 끊기면 다시 걸고 자리는 한동안 유지). 공장은 이 전화로 계측 · 상태 · 명령 결과 · 통신 상태 · 요청 응답을 올려 보내고(QoS 1) [29], 같은 전화로 사무실이 남긴 작업 요청서(QoS 1)와 분석 경고(QoS 0)를 받아 옵니다 [30]. 브리지로 들어온 쪽지에는 게시판의 계정별 권한이 적용되지 않으므로, 브리지 설정의 「들여올 칸」 목록이 그대로 거름망입니다. 요청서는 사무실에서 공장 쪽으로 흐르지만 그 전화를 건 쪽은 공장입니다. 「누가 연결을 열었나」와 「서류가 어느 쪽으로 가나」는 다른 이야기입니다.

감시 기록도 같은 원칙으로 공장 쪽 에이전트가 접수처로 밀어 보냅니다(Prometheus remote write, HTTP) [31]. 접수처의 감시 장치(Prometheus)는 요청 창구 [35], 적재기 [109], 원본 사본 [110]의 지표도 긁습니다. 접수처 우편함도 익명을 막고, 계정 여섯(브리지 · 적재기 · 요청 창구 · 사무실 수집기 · 보기 전용 · 감시)이 각자 허락된 칸만 씁니다.

우편함에 도착한 값은 두 곳으로 갑니다. 하나는 접수처의 **적재기**(Bento)입니다. 적재기는 우편함의 계측 · 상태 칸을 구독해(MQTT, 세션을 남기지 않는 접속) [32] 묶음으로 **원본 사본**(InfluxDB)에 씁니다(HTTP, 라인 프로토콜) [33]. 여기서 UNS 메시지는 시계열 행으로 바뀝니다. 계측은 「공정 원본」 측정값으로, 사이트 · 구역 · 라인 · 설비 · 센서 · 품질이 꼬리표, 값 · 스캔 순번 · 설비 시각이 내용이 되고, 시각은 엣지가 잰 시각을 그대로 씁니다. PLC 상태는 「PLC 상태」 측정값이 됩니다. 품질 BAD는 행을 만들지 않아 빈칸으로 남고, STALE은 품질 꼬리표를 단 채 기록합니다(사무실 Kafka에는 STALE이 들어가지 않는 것과 다릅니다). 명령 결과와 요청 응답은 사본에 넣지 않습니다. 사본은 일주일 남짓 보관합니다. 같은 적재기의 둘째 스트림은 우편함의 상태 칸을 감시 지표로 바꿉니다 [108].

원본 사본에 쓰는 열쇠는 처음 만들 때의 **관리자 토큰**입니다. 모든 권한을 가진 토큰이고, 같은 관리자 계정으로도 쓸 수 있습니다. 사무실이 쓰는 것은 초기화 스크립트가 따로 만든 읽기 전용 계정(InfluxQL 호환 조회)입니다. 사무실의 AI와 사람은 공장에 직접 묻지 않고 이 사본만 읽습니다. 우편함에 도착한 값의 다른 하나는 사무실 쪽 심부름꾼에게 갑니다.

5장 사무실의 장부와 실시간 검사관

사무실의 심부름꾼은 **IT 수집기**(Bento)입니다. 컨테이너 하나 안에 스트림이 여럿 돕니다. ① 접수처 → Kafka, ③ 표시 경보 → 접수처, ④ 분석 경보 → Alertmanager, ⑤ PLC 상태 → 억제 상태, ⑥ Alertmanager 알림 → 경보 기록 · 표시, ⑦ 승인 요청 → 요청 창구, ⑧ MES 주문 → 요청 기록(②번은 비어 있음). ①이 담장을 지나 접수처 우편함을 구독하고(MQTT, 세션을 남기지 않는 접속) (36), 쪽지 종류에 따라 장부 Kafka의 세 칸에 나눠 적습니다 (37). 계측값은 품질 GOOD인 것만 sensor.telemetry.raw 칸에, 칸 이름 조각(사이트 · 라인 · 설비 · 센서)을 쪽지 안에 풀어 넣고 설비 이름을 열쇠로 적습니다. PLC 상태 · 설비 상태 · 명령 결과는 plant.status 칸에, 요청 응답은 request.responses 칸에 요청 번호를 열쇠로 적습니다. 세션을 남기지 않으므로 수집기가 꺼져 있던 동안의 값은 우편함이 모아 두지 않고, Kafka에는 빠진 채 접수처 원본 사본에만 남습니다.

Kafka(KRaft 단일 노드)는 녹화 서버이자 은행 원장입니다. 들어온 순서대로 번호를 붙여 디스크에 적고, 누가 읽어도 지우지 않습니다. 칸은 계측 원본(sensor.telemetry.raw), 정리본(sensor.telemetry.clean), 이상 점수(sensor.anomaly.score), 경보(sensor.alerts), PLC 상태(plant.status), 표시 경보(alerts.display)가 짧게 보관되고, 승인 요청(request.approved), 요청 사건(request.events), 요청 응답(request.responses), 감사 사본(audit.copy)이 더 길게 보관됩니다. 읽는 쪽은 저마다 「어디까지 읽었나」 책갈피(오프셋)를 Kafka에 맡겨 두므로, 여럿이 서로 모른 채 각자 속도로 읽고, 멈췄다 돌아와도 이어 읽고, 필요하면 보관 기간 안에서 처음부터 다시 읽을 수 있습니다. 전달은 「최소 한 번」이라 같은 쪽지가 두 번 올 수 있고, 중복은 읽는 쪽이 거릅니다. 그러니까 「MQTT 대신 Kafka」가 아니라 「MQTT 다음에 Kafka」입니다. 공장은 생중계로 지금 값을 나누고, 사무실은 녹화로 오래 남겨 여러 번 씁니다. 단, 사무실의 최종 기록(정본)은 Kafka가 아니라 업무 DB PostgreSQL입니다. Kafka는 통로이자 사본입니다.

장부에 새 줄이 적히는 순간 **실시간 검사관**(Flink)이 읽습니다. 통에 다 받은 뒤 검사하는 것이 아니라 흐르는 물줄기에 체를 대는 방식입니다. 규칙 체는 SQL 잡 셋입니다 (39). 쪽지의 측정 시각을 사건 시각으로 쓰고 조금 늦게 오는 것까지 기다리며, 읽을 수 없는 쪽지는 버립니다. 규격 체는 등록부에서 만든 상하한을 넘은 측정마다 「심각」 경보를 냅니다. Z-Score 체는 센서마다 최근 값들에 비해 크게 튀는 일이 거듭되면 「경고」를 냅니다. 순서 체(CEP)는 라인 단위로 「교반기 전류가 상한을 넘은 뒤 곧 진동이 상한을 넘었나」를 보고 「심각」 경보를 진동 센서 이름으로 냅니다. 여기서 **경보**가 태어납니다. 경보 쪽지에는 시각 · 사이트 · 라인 · 센서 · 값 · 종류 · 심각도 · 탐지기 · 설명이 들어가고, 탐지기는 규칙 · Z-Score · CEP · AI 모델 · 업무 서비스 가운데 하나입니다.

또 하나는 AI 모델 체(ONNX 잡)입니다 (40). 먼저 **정리본**을 만듭니다. 시각이 거꾸로 왔거나 겹친 쪽지는 버리고, 빈 곳은 짧으면 앞뒤를 직선으로 잇고 길면 마지막 값을 이어 쓰며 「메운 값」 표시를 붙입니다. 다음으로 라인마다 같은 스캔의 센서 값을 한 줄로 맞추고, 최근 몇 줄의 창을 서막의 오토인코더에 넣어 「요약했다 다시 펼친 결과」가 원래와 얼마나 다른지 **이상 점수**를 매깁니다. 점수가 학습 때 정한 기준을 넘으면 「다변량」 이름의 경보를 냅니다. 점수 쪽지에는 많이 어긋난 센서가 함께 적힙니다. 정리본 · 이상 점수 · 이상 경보는 sensor.telemetry.clean · sensor.anomaly.score · sensor.alerts 칸에 적힙니다 (42). 추론은 Flink 작업자 안에서 onnxruntime으로 합니다. 원본의 빈칸은 그대로 두고 「메운 값」은 정리본에만 둡니다. 실제로 잰 값과 추정한 값을 구별해야 하기 때문입니다.

검사관은 진행 위치를 주기적으로 체크포인트로 저장해 다시 켜지면 그 위치부터 이어 갑니다. 다만 「정확히 한 번」은 검사관 내부 상태에만 해당하고, Kafka로 내보내는 경보는 「최소 한 번」이라 복구 직후 같은 경보가 두 번 나갈 수 있습니다. 그런 중복은 뒤의 알림 비서 묶기와 AI 사건 접수의 중복 제거가 받아냅니다.

PLC 상태 칸도 계속 읽힙니다. 업무 서비스는 PLC의 최신 상태를 기억해 두었다가, 뒤에서 「명령대로 실제로 바뀌었나」를 확인할 때 씁니다 (69). 수집기 ⑤번 스트림은 상태를 읽어 (48) 알림 비서(Alertmanager)에게 **억제 상태**를 경보 모양으로 알립니다(HTTP) (49). 정비 모드가 켜지면 「사용 중지(Out of Service)」, PLC 프로그램이 멈추면 「공장 정지」, 히터 · 냉각기 · 펌프 · 교반기 중 운전 명령이 꺼진 설비가 있으면 그 설비의 「설계상 억제(Suppressed by Design)」입니다. 켜져 있는 동안은 끝 시각을 멀리 미뤄 두고, 풀리면 즉시 끝냅니다. 경보 관리 표준 ISA-18.2의 용어입니다.

사무실의 정본은 **업무 DB**(PostgreSQL)입니다. DB가 둘인데, plant DB에는 등록부, 작업 요청과 요청 사건, 경보 묶음과 경보 사건, 감사 기록, 회사 사실값(MES · ERP · CMMS)과 정비 작업 이력이 있고, ai DB에는 AI의 받은 경보 원본 · 사건 · 분석 실행 · 대응안 · 체크포인트 · 지식 후보 기록이 있습니다. 요청 · 요청 사건 · 경보 사건 · 감사 · 변경 관리 표는 한 번 쓰면 고치거나 지울 수 없게 막아 두었고(트리거와 권한 회수), 경보 묶음만 수집기 계정이 고칩니다. 요청의 「지금 상태」는 따로 고치지 않고 마지막 사건으로 정합니다. 계정은 수집 · 발송 · MES · AI · 읽기 전용으로 나뉘어 각자 필요한 권한만 있습니다. 초기화 스크립트는 빈 볼륨의 첫 기동에만 돌고, 다시 돌려도 사람이 바꾼 값을 덮어쓰지 않습니다.

감시 구성을 켠 날에는 검사 결과(정리본 · 점수 · 경보)가 이력 적재기(Bento)를 거쳐 (62) 결과 기록장(InfluxDB)에 쌓입니다(HTTP, 라인 프로토콜) (63). 그래프 화면(Grafana, 로그인 없이 보기 전용)의 대시보드는 넷입니다. 결과 기록장을 그리는(Flux) (64) 「공정 트렌드」 · 「AI 이상 점수」 · 「알람 이력」, 그리고 감시 장치를 그리는(PromQL) (111) 「인프라」입니다. 접수처 원본 사본도 데이터 원천으로 연결은 해 두었지만 그것을 읽는 패널은 없습니다 (65).

사무실 쪽 건강검진 의사(Prometheus)는 접수처의 감시 기록을 통째로 가져오고(federate, HTTP) (66), 장부 밀림을 재는 kafka-exporter와 컨테이너 자원을 재는 cAdvisor (67), Flink (112), IT 수집기 (113), 결과 기록장 (114)을 긁습니다. kafka-exporter는 Kafka에 붙어 소비 그룹별 책갈피와 밀림을 읽습니다 (115). 업무 DB · ZooKeeper · Alertmanager · Neo4j · AI 업무 도우미 · 업무 서비스 · AI 화면은 감시 대상이 아닙니다. 경보 규칙은 장부 밀림, Flink 체크포인트 실패 · 반복 재시작, 공장 → 접수처 브리지 끊김, 수집기 입력 멈춤, 브로커 연결 해제 급증, 브로커가 쪽지를 버림, 엣지 디스크 큐 적체, 계측 원본 유입 멈춤, 서비스 응답 없음입니다. 이 **인프라 경보**는 알림 비서의 기본 경로로 가는데 (68), 그 경로의 수신자에는 보낼 곳이 하나도 설정되어 있지 않습니다. 메일도 메신저도 가지 않고 알림 비서 화면에서만 보입니다. 이들은 공장 값이 아니라 시스템 자체를 봅니다.

6장 베어링이 닳기 시작한다

어느 오후, 교반기의 베어링이 닳기 시작합니다. 훈련용 공장에서는 강사가 같은 PC에서 담장을 거쳐(HTTP, 강사 계정) (103) 「베어링 마모」 고장을 넣고 (5), 고장 엔진이 물리 계산에 마찰을 더합니다 (3). 소리도 연기도 없습니다. 전류가 먼저 상한을 넘고, 조금 뒤 진동이 따라 오릅니다. 이 고장은 베어링 교체 전에는 사라지지 않습니다(아주 긴 기본 기한이 지나면 저절로 사라지도록 되어 있습니다). 이 값들은 1~5장의 길을 그대로 따라 장부까지 갑니다.

검사관의 체가 울립니다. 규격 체가 상한을 넘은 전류와 진동 측정마다 경보를 내고, 순서 체에 「전류가 먼저, 진동이 나중」이 걸리며, 값이 급히 변하는 동안 Z-Score 체가 울리기도 합니다. 검사관은 이것들을 경보 쪽지로 sensor.alerts 칸에 적습니다 (41). 이 칸이 갈림길입니다.

첫째 갈래는 **알림 비서**(Alertmanager)입니다. 수집기 ④가 경보를 읽어 (46) 비서의 형식으로 바꿔 넘깁니다(HTTP) (47). 경보 쪽지는 여기서 꼬리표 묶음으로 이름을 바꿉니다. 출처(공정) · 경보 이름 · 규칙(종류:센서) · 설비 · 센서 · 심각도 · 탐지기이고, 설비 이름은 서막의 태그 → 설비 지도로 붙입니다(지도에 없으면 라인 이름). 끝 시각을 조금 뒤로 적어 두므로, 같은 경보가 계속 오면 늘어나고 조용해지면 비서가 스스로 해제합니다. 비서는 공정 경보를 설비 · 규칙 · 경보 이름이 같은 것끼리 한 **묶음**으로 만들고, 새 묶음과 해제를 바로 알리며, 계속 울리는 묶음은 오랜 간격을 두고 다시 알립니다.

억제의 범위는 정확히 이렇습니다. 「사용 중지(정비 모드)」와 「공장 정지」는 설비를 가리지 않고 **라인 전체의 공정 경보**를 누릅니다. 「설계상 억제」만 **같은 설비**의 경보를 누릅니다. AI 모델 경보와 명령 불일치 경보는 설비 칸에 라인 이름이 들어가므로 설비별 억제에는 걸리지 않습니다. 경보가 한꺼번에 쏟아지면(경보 홍수) 운전원이 정작 중요한 한 건을 놓친다는 문제를 다루는 국제 표준이 ISA-18.2(경보 관리)이고, 이 공장의 경보 기록은 그 표준의 상태 가운데 「사용 중지」와 「설계상 억제」를 씁니다(운전원이 잠시 미뤄 두는 「보류」는 없습니다).

새 묶음이 생기거나 풀리면 비서가 수집기 ⑥에 알리고(HTTP 웹훅) (50), ⑥은 설비 · 규칙별 **경보 묶음**을 「진행 중」으로 만들거나 갱신하고 「표시함」 사건을 남기며, 해제되면 「해제됨」으로 닫습니다. 억제가 시작되고 풀린 것도 사건으로 남깁니다(SQL) (51). 억제로 눌린 공정 경보는 경보 기록에 남지 않고, Kafka와 AI 사건 접수에만 남습니다. ⑥은 새로 뜬 공정 경보만 한 줄짜리 **표시 경보**(시각 · 심각도 · 센서 · 종류)로 바꿔 alerts.display 칸에 올리고 (52), 수집기 ③이 그것을 읽어 (53) 접수처 우편함의 표시 칸에 「마지막 한 장 남기기」로 붙입니다(MQTT) (54). 그 한 장이 공장이 건 전화를 타고 게시판으로 내려와 (30) 운전실 화면의 「분석 경고(참고)」 칸에 뜹니다 (23). 마지막 한 장만 남기므로 화면에는 최근 분석 경고 하나만 보입니다. 공장 스스로 판단하는 알람과 칸을 나눠, 사무실 분석은 참고 정보라는 점을 분명히 합니다.

둘째 갈래는 **AI**입니다. 업무 서비스(business 컨테이너)의 접수 스레드가 자기 책갈피로 같은 경보 칸을 처음부터 빠짐없이 읽습니다 (72). 쪽지마다 칸이 다 있고 값이 숫자인지 검사하고, 통과하지 못한 것도 「거부됨」으로 원본 그대로 보관합니다. 쪽지 내용 전체의 지문(SHA-256)이 같으면 같은 경보로 보고 다시 세지 않습니다. 그다음 **상관 묶음**으로 **사건**을 만듭니다(SQL) (73). 교반기 묶음(교반기 전류 · 진동의 규격 · 순서 · Z-Score 경보), 반응기 열 묶음(반응기 · 재킷 온도, 냉각수 회수 온도 · 유량, 스트레이너 차압), 압력 묶음(두 압력계 · 반응기 액위의 상한 경보)은 각각 한 사건 후보가 되고, 나머지는 센서와 종류별로, AI 모델 경보와 명령 불일치 경보는 따로 묶입니다. 같은 묶음의 경보가 사건 구간 가까이에 오면 그 사건에 붙고 구간이 늘어납니다. 경보가 붙을 때마다 사건 버전이 오르고, 새 조합의 경보나 더 이른 경보가 붙으면 「검토 버전」도 올라, 그때 기다리던 검토 카드는 무효가 됩니다.

저장을 먼저 하고 책갈피를 나중에 옮기는 순서라, 처리 도중 프로세스가 갑자기 꺼져도 경보를 잃지 않습니다. 다만 저장하다 DB 오류가 나면 그 경보는 다음 쪽지의 책갈피와 함께 지나가 버릴 수 있습니다(알려진 한계). 알림 비서가 눌러 둔 경보도 이쪽에는 모두 들어옵니다. 규격 한계를 넘지 않은 경보만 있는 사건은 「참고 신호」로 표시되고, 자동 분석 대상이 아닙니다.

7장 AI가 원인을 가린다

담당자는 브라우저로 **단일 운영 포털**(Vue 웹 앱, nginx)을 열어 둡니다 (74). 업무 요청은 AI 업무 도우미로, 사람의 설비 조작은 전용 접점으로 넘깁니다. 쓰기 요청이 실패해도 자동 재전송하지 않습니다. 왼쪽의 운영 현황·온톨로지·문서·지식·처리 이력 메뉴로 이동합니다. 운영 현황에서 설비와 사건의 진행을 나란히 봅니다. 메뉴를 옮겨도 진행 중인 사건과 검토 의견은 유지합니다. 원인 분석 중에는 실제 도구와 지금 하는 일을 한두 줄로 자동 갱신하고, 분석 결과와 원인 비교·정비 대안은 상세 근거에서 확인합니다. 감지·원인 분석·담당자 검토·승인/반려·정비·복구 확인을 담은 전체 흐름 그림은 항상 펼쳐져 있고, 실제 저장 상태를 색으로 표시합니다. 별도 BPM 엔진이나 단계를 실행하는 버튼은 아닙니다. 정상 진행에서 사용자는 이상을 발생시키고, 대응안을 검토해 의견을 작성한 뒤 승인 또는 반려만 합니다. 온톨로지는 설비·센서, 이상·원인, 조치·판단, 문서·근거, 성과 지표를 고정된 열에 정리합니다. 선택한 항목의 직접 연결을 강조하고 지도 아래에서 관련 근거를 따라갑니다. 원본 근거와 상세 기록은 필요할 때 펼칩니다.

설비 영역은 반응기 공정도와 주요 센서·추세·인터록 상태를 보여 줍니다. 데이터는 업무 도우미가 접수처 원본 사본에서 읽은 PLC 상태와 최근 계측이고 (118), 새 스캔이 끊기면 갱신 확인 안내가 뜹니다. 이상 발생 후 사건은 자동으로 선택되며 분석과 대응안 작성은 승인 대기까지 자동 진행합니다. 추가 운전 조작은 포털 안의 설비 제어 패널로 열고 비밀번호 입력 없이 운전원으로 연결됩니다. 수집·분석 연결 상태는 펼쳐 볼 수 있습니다. 공정 조회, 최근 GOOD 센서와 Flink 작업 상태를 확인합니다 (88). 이 작업 상태는 사람에게 보여 주는 조회이며 AI 분석 도구의 근거를 대신하지 않습니다.

화면은 짧은 간격으로 사건 목록 · 설비 상태 · 모델 연결 상태를 묻습니다(HTTP) (75). AI 업무 도우미(knowledge: FastAPI + LangGraph)는 업무 서비스가 만든 사건을 AI DB에서 읽어 보여 주고(SQL) (77), 백그라운드에서 자동 분석할 사건을 고릅니다 (78). 조건은 이렇습니다. 사건이 「접수됨」이고, 새 종류의 경보가 한동안 더 붙지 않았고, 경보가 여러 건 쌓였고, 최근에도 경보가 오고 있으며, 분석한 적이 없거나 실패 · 중단 뒤 검토 버전이 올랐고, 규격 한계나 순서 경보가 있으며(Z-Score나 AI 모델 경보만으로는 정비 판단을 열지 않음), 증상에 정비 결정이 이어져 있고, 언어 모델이 설정되어 있을 것. 건너뛴 사건에는 이유를 남깁니다. 고장 직후에는 전류 · 진동 · 순서 경보가 시차를 두고 줄줄이 붙어서, 첫 경보만 보고 시작하면 분석 도중에 사건의 모양이 바뀌기 때문에 기다리는 것입니다. 반려되었거나 해결되지 않은 사건을 다시 분석하는 것은 사람이 시작합니다.

분석은 세 걸음입니다. 조사 → 사람 검토(여기서 멈춤) → 실행. 사건마다 동시에 하나만 돕니다. AI는 탐정처럼 도구로 단서를 모읍니다 (81). 경보 요약, 설비 · 문서 · 상하한, 센서 관측, 고장 온톨로지 추적, 매뉴얼 의미 검색, 선례(같은 사건의 앞선 카드 · 반려 사유 · 현장 소견은 근거로, 다른 사건은 참고로), 정비 대안 평가입니다. 필수 도구를 모두 부르기 전에는 결론을 낼 수 없습니다.

근거는 이렇게 모입니다. 센서 관측은 접수처 원본 사본에서 읽습니다(InfluxQL, 읽기 전용) (86). 사건 구간(첫 경보 조금 전부터 마지막 경보 조금 뒤까지)과 지금 최근 값을 따로 봅니다. 첫 경보가 울린 순간의 값이 중요합니다. 시간이 지나면 전류도 진동도 다 높아져 무엇이 먼저였는지가 지워지기 때문입니다. 같은 자리에서 지식 지도의 설비 관계를 읽어(Bolt) (87) 경보 센서의 설비, 거기 설치된 설비 · 공급 설비 · 상위 설비, 그 설비들의 문서 전문과 센서 상하한을 가져옵니다.

지식 지도에는 이런 것이 이어져 있습니다. 경보 모양 → 증상 → 고장 모드 → 원인 · 확인 관측, 고장 모드 → 영향받는 부품, 증상 → 결정 → 규칙 · 대안 → 실행 단계 → 작업 정의, 대안 → KPI 영향, 문서 → 절. 매뉴얼은 교반기 정비 · 설비 맥락 · 냉각수 · 근거 정책 · 고장 모드 · 정비 정책 · 교반기 대응 · 압력 대응 · 센서 맥락 · 열 대응입니다. AI는 지식

지도에서 용의자를 찾습니다(Bolt) (80) (83). 교반기 진동 증상의 용의자는 넷, 베어링 마모 · 축 정렬 불량 · 임계 속도 부근 운전 · 반응기 액위 부족이고, AI는 넷 모두를 「지지 · 반박 · 알 수 없음 · 해당 없음」 중 하나로 평가해야 합니다. 「확정」이라는 답은 없습니다. 베어링 상태는 센서로 잴 수 없어 확정은 분해 점검이기 때문입니다.

매뉴얼은 뜻으로 찾습니다. 질문을 숫자 지문으로 바꿔(OpenAI 임베딩, HTTPS) (85) 서막에서 붙여 둔 지문과 가까운 절을 고르고, 그중 증상과 이어진 절을 앞에 세웁니다. 언어 모델(OpenAI gpt-6-luna, Responses API, HTTPS, 재시도 없음)은 이 단서들을 읽고 정해진 서식으로 정리합니다 (79). 결과는 「근거를 갖춘 제안」 또는 「근거 부족」 둘 중 하나이고, 인용은 이번에 조회된 문서만 쓸 수 있으며, 남은 불확실성을 반드시 적어야 하고, 서식이 틀리면 실패입니다. 카드 본문은 「관측 · 원인 판단 · 대안 비교 · 정비 계획 · 승인 시 영향」이고, 고른 대안의 절차 문서를 꼭 인용해야 합니다. 결론은 「전류가 먼저 상한을 넘었다 → 베어링 마모 지지, 축 정렬 불량 반박」입니다.

고칠 방법의 손익은 언어 모델이 아니라 **판단 엔진**이 셉니다 (84). 같은 입력이면 늘 같은 답을 내는 안전한 식 계산기이고, 값이 하나라도 없으면 「계산 불가」입니다. 입력은 지금 관측값과 상하한, 지금 명령 상태, 대안의 성격, AI의 원인 평가, 그리고 업무 DB의 회사 사실값(MES의 생산 가치 · 납기 · 배치, ERP의 예비품 재고 · 단가, CMMS의 인건비 · 위험 비용)입니다. 계산기는 먼저 **자격**을 봅니다. 치료하는 고장 모드가 「지지」면 원인 대응 자격, 원인을 좁히지 못했으면 점검형 자격입니다. 다음으로 **규칙**에 걸린 대안을 빼고, 남은 대안을 KPI 합계로 줄 세웁니다. 「돈으로는 1위지만 규칙으로 제외」된 대안도 숨기지 않고 함께 돌려줍니다. 마지막으로 **뒤집힘**, 곧 어떤 사실값이 얼마나 바뀌면 1위가 바뀌는지를 찾습니다. KPI는 균형 성과표(BSC)의 관점에 걸린 생산 손실 · 납기 지연 · 품질 손실 · 정비 비용 · 설비와 안전 위험입니다. 최종 대안은 AI가 자격 있는 대안 가운데서 고릅니다. 기본은 손익 1위이고, 앞선 반려 사유가 있으면 다른 대안을 고르되 이유를 적어야 합니다.

결정은 셋입니다. **교반기 진동 정비**(지금 정지해 베어링 교체 · 배치 끝까지 운전 뒤 교체 · 지금 축 정렬 · 배치 뒤 축 정렬 · 둘 다 점검; 진동이 매우 크면 배치를 기다리는 안, 예비 베어링이 없으면 베어링 안, 교반기가 이미 멈췄으면 배치를 기다리는 안을 뺌), **냉각 회복**(운전 중 스트레이너 전환 · 세척, 감속 뒤 전환, 정지하고 재킷 화학 세정, 감속하며 버티기, 전체 점검; 상한을 넘은 채 계속 운전하는 안과 세정제가 없을 때의 세정 안을 뺌), **고압 · 인터록 대응**(압력계 교정 · 밸브 정비 · 인터록 우회 뒤 재가동 · 둘 다; 인터록 우회는 실행 단계가 없고 규칙으로도 뺌). 결정마다 「고쳐졌다」고 볼 회복 기준(센서 조건을 일정 시간 유지)이 함께 있습니다. 베어링 사건에서 계산기는 「배치 끝까지 운전 후 교체」를 1위, 「지금 교체」를 2위로 세웠습니다. 지금 멈추면 배치를 버리는 손실이 들고, 배치를 마치면 그동안의 축 고착 위험이 붙는 셈이 비교된 것입니다.

분석 과정과 「사람 검토 대기」 지점은 AI DB에 저장됩니다(SQL, LangGraph 체크포인트) (82). 도우미가 다시 켜지면 분석 도중이던 실행은 「중단됨」이 되어 새로 시작해야 하지만, 사람 검토를 기다리던 실행은 저장된 대기 지점이 남아 있어 카드 기한 안이면 그대로 승인 · 반려할 수 있습니다. 중단 · 실패한 실행은 저장된 상태만 읽어 정리하는 복구 기능이 있고, 설비 명령을 다시 보내지는 않습니다.

8장 사람이 결정한다

운영 화면에 **예상 원인과 권장 정비**가 뜹니다. 담당자는 승인 시 영향과 주의 사항을 확인하고 검토 의견을 작성합니다. 상세 근거를 펼치면 원인 판단 표, 대안별 KPI와 제외 사유, 회사 사실값, 작업 순서와 회복 기준을 확인할 수 있습니다. 「배치가 끝나면 교반기를 멈추고, 정비 모드로 잠근 뒤, 정비팀이 베어링을 교체하겠습니다. 승인하시겠습니까?」 카드에는 검토 기한이 있어, 지나면 승인 버튼이 꺼지고 반려만 남습니다. AI는 여기서 멈춰 사람을 기다립니다. 실행 단계는 이 카드를 만들 때 이미 고정됐습니다. 그 순간의 명령값을 넣고 작업 정의의 범위 안인지도 그때 확인했습니다.

담당자가 메모를 적어 승인하면(HTTP) (76), AI의 「실행」 걸음이 **승인 재검사**로 넘어갑니다 (89). 그 순간 한 번 더 확인합니다. 카드가 기한 안인지, 사건의 검토 버전이 그대로인지, 공정 상태를 지금 조회할 수 있는지(조회할 수 없으면 「조치하지 않음」), 스위치 · 인터록 · 정비 모드의 지문이 카드를 만들 때와 같은지, 적용 문서와 설비 관계(지식 지문)가 그대로인지 (119). 정비 계획이면 판단 엔진을 지금 사실값과 관측으로 다시 돌려, 고른 대안이 여전히 자격이 있고 규칙에 걸리지 않는지도 봅니다 (120). 하나라도 어긋나면 승인을 받지 않고(충돌 응답) 그 분석 실행은 실패로 끝나며, 다시 분석하는 것은 사람이 시작합니다.

통과하면 정비 계획이 실행기로 넘어갑니다 (92). 계획을 승인했다고 작업 요청이 한꺼번에 생기지는 않습니다. 요청서는 실행기가 단계마다 한 장씩 씁니다. AI가 「현장 점검 요청」을 낸 경우라면, 승인해도 설비 명령은 하나도 나가지 않고 점검 요청을 기록한 뒤 「정비 대기」가 됩니다. 같은 결정을 다시 보내면 처음 결과를 그대로 돌려주고, 다른 결정을 보내면 거절합니다. AI가 낸 카드는 이 검토 단계를 거쳐서만 승인할 수 있습니다.

사유를 적어 반려하면 요청은 한 건도 나가지 않고 사건은 「미해결」이 됩니다. 사람이 다시 분석을 시작하면 AI는 그 반려 사유를 선례로 읽고 새 카드를 냅니다. 시연에서 「고착 위험이 더 크다」는 사유로 반려했더니, 새 카드는 「지금 교체」를 고르면서 손익 1위와의 차이를 숨기지 않고 적었습니다.

그림을 보면 AI 업무 도우미에서 설비로 곧장 가는 화살표는 하나도 없습니다. 다만 AI 화면의 「설비 제어 패널 열기」는 운전실 화면 FUXA를 창 안에 띄우는 기능이라, 그때 브라우저가 라우터와 문지기를 거쳐 FUXA에 직접 접속합니다 (101). 그러니 사람은 AI 화면 안에서도 FUXA 계정으로 설비를 조작할 수 있습니다. 그것은 3장의 운전원 명령이지 AI의 명령이 아닙니다.

9장 다시 설비로: 세 겹의 검사

이제 이야기가 방향을 바꿔 사무실에서 공장으로 내려갑니다. **정비 계획 실행기**(한 걸음씩 나아가는 상태 기계)의 단계는 네 종류입니다. PLC에 명령을 요청하는 단계, 정비팀에게 작업을 요청하는 단계, 운전원만 할 수 있는 일(예: 인터록 리셋)이 끝나 조건이 참이 되기를 기다리는 단계, 설비 시간을 기다리는 단계. 베어링 계획은 이렇습니다. 배치 끝까지 운전(실행기 안에서 설비 시간을 기다림, 요청서 아님) → 교반기 정지 → 정비 모드 켜기 → 베어링 점검 · 교체(정비팀) → 작업 완료 확인 · 격리 해제(정비팀의 잠금 해제 작업) → 교반기 재기동. 앞 단계가 끝났다고 판정돼야 다음 요청서가 나가고, 정해진 시간 안에 결과가 확인되지 않으면 그 단계는 실패입니다. 단계가 바뀔 때마다 상태를 AI DB에 저장하고, 다시 켜지면 이미 보낸 요청은 요청 번호로 결과만 다시 읽을 뿐 다시 보내지 않습니다.

작업 요청은 먼저 업무 DB(PostgreSQL)에 정본으로 태어납니다. 요청 번호 · 작업 정의 · 설비 · 값 · 요청자(ai-ops) · 승인자 · 만든 시각 · 사건 요약과 사건 · 대응안 번호, 그리고 「승인됨」 사건과 감사 기록(AI가 제안, 사람이 승인)이 함께 들어갑니다(SQL) (93). 이 표들은 한 번 쓰면 고치거나 지울 수 없습니다. 장부 Kafka의 request.approved 칸에는 내용 없이 **요청 번호만** 적힙니다 (94). 내용의 주인은 하나여야 하고, 그것이 고칠 수도 지울 수도 없는 업무 DB이기 때문입니다.

그림에는 대응안 처리 쪽이 직접 요청서를 쓰고 번호를 알리는 길도 그려져 있습니다 (90) (91). 「교반기만 멈추기」, 「냉각기 켜기」 같은 한 건짜리 조치를 위한 길입니다. 하지만 지금 AI가 낼 수 있는 대응안 종류는 「정비 계획」과 「현장 점검 요청」 둘뿐이라, 이 길은 코드에 남아 있을 뿐 지금은 쓰이지 않습니다.

사무실 수집기의 ⑦번 스트림이 그 번호를 읽고 (55) 정본에서 내용을 꺼냅니다. 마지막 사건이 「승인됨」이고 아직 「보냄」이 없는 요청만 보내며, 「보냄」과 감사 기록을 먼저 남긴 뒤(SQL) (56) 담장을 지나 접수처의 요청 창구로 딱 한 번 보냅니다(HTTP, 출입 토큰, 재시도 없음) (57). 다시 보내지 않는 이유는 「순조롭지 않을 때」에 있습니다. 창구의 대답, 곧 ⓐ **게이트웨이 응답**(받음 · 거절과 사유)을 정본에 적고, 시간 초과나 연결 실패는 「게이트웨이에 닿지 않음」 거절로 적습니다. 사본은 장부의 request.events · audit.copy 칸에도 남깁니다 (58).

검사 1 — 접수처 요청 창구(Node-RED, 편집기 꺼짐). 출입 토큰이 맞는가, 서식이 약속된 틀(JSON Schema, 정해진 칸 밖은 금지)과 같은가, 허락된 작업 · 그 작업의 설비 · 허락된 값 범위인가, 요청자 번호와 종류의 짝이 명단에 있는가, 승인자가 명단에 있는가, 너무 오래되지 않았는가, 같은 요청 번호가 또 오지 않았는가, 우편함과 연결돼 있는가. 공장의 운전 모드는 여기서 보지 않습니다. 하나라도 걸리면 거절 사유를 돌려줍니다. 모두 통과하면 요청서에 만료 시각을 적고, 「제때 전달되지 않으면 폐기」를 붙여(MQTT 5 메시지 만료) 우편함의 요청 칸에 넣습니다(QoS 1) (34). 우편함의 받음 확인(PUBACK)이 와야 「받음」을 대답하고, 오지 않으면 거절하며 중복 기억에서도 지웁니다. 요청서는 공장이 건 전화를 타고 공장 게시판으로 내려갑니다 (30).

바깥에서 요청할 수 있는 작업은 등록부의 작업 정의뿐입니다. PLC 명령으로는 교반기 정지 · 재기동, 냉각기 켜기, 반응기 온도 설정, 펌프 속도 · 정지 · 재기동, 정비 모드 켜기가 있고, 현장 작업으로는 스트레이너 예비 바스켓 전환과 대기 바스켓 세척(운전 중 가능), 재킷 화학 세정 · 베어링 점검 교체 · 축 정렬 · 배출 밸브 정비(정비 모드 필요), PT-101 비교 교정(계기 격리), 잠금 해제(정비 모드 필요, 끝에 정비 해제 키)가 있습니다. 값의 범위는 두 겹이라 요청 쪽이 PLC보다 좁습니다. 운전 모드 변경 · 인터록 리셋 · 히터 · 밸브 개도 · 정비 해제는 요청할 수 없습니다.

검사 2 — 공장 접수 담당(엣지 Node-RED의 둘째 탭, 자기 계정, 세션을 유지하는 MQTT 5 연결). 요청서와 운전실의 결정, PLC의 지금 상태를 받아 (20) 다시 검사합니다. 서식을 처음부터 다시 보고, 같은 요청이 또 왔나 → 허락된 작업 · 설비 · 값인가 → 만료되지 않았나 → **지금 공장이 받을 수 있는가**(PLC 프로그램이 돌고 있고 현장 통신이 살아 있나, 현장 모드가 아닌가, 정비 모드가 필요한 현장 작업인데 정비 모드가 꺼져 있지 않은가, 기계를 움직이는 작업인데 정비 중은 아닌가) → 모드별 처리 순서로 봅니다.

원격 자동 모드면 바로 받습니다. 기계를 움직이는 작업은 PLC 명령으로, 현장 작업은 곧바로 정비팀에게 갑니다. 원격 수동 모드면 운전실 화면의 **대기 목록**(「마지막 한 장 남기기」 칸, 대기 건수 · 맨 앞 요청 · 「[AI ai-ops]」로 시작하는 원인 요약 · 대기 시간)에 올리고 운전원의 결정을 기다립니다. 화면의 수락 · 거부 버튼은 맨 앞 요청 하나에 대한 것입니다. 운전원이 수락하면 그 결정이 게시판을 거쳐 돌아오고 (24) (20), 접수 담당은 모드와 정비 상태를 다시 본 뒤, 기계를 움직이는 작업에는 **수락한 시각부터 새로 센** 만료와 운전원 수락 표시를 붙이고, 현장 작업은 바로 정비팀에게 넘깁니다. 너무 오래된 결정, 남겨진(retain) 결정, 대기 목록에 없는 요청에 대한 결정은 무시합니다. 결과는 단계마다 **응답 쪽지**(요청

번호 · 단계 · 결과 · 사유 · 시각)로 게시판에 붙습니다(MQTT) 21. ⓑ **접수 응답**은 자동 수용 · 운전원 대기 · 거절과 사유이고, ⓒ **응답**은 운전원(수락 · 거부 · 시간 초과), PLC(수용 · 거절과 응답 코드), 현장(완료 · 고장 없음 · 그 밖) 입니다.

기계를 움직이는 단계(교반기 정지 · 정비 모드 켜기 · 교반기 재기동)면 접수 담당이 PLC 명령을 게시판의 그 설비 요청 칸에 붙이고 21, 엣지가 받아 19 PLC의 외부 요청 채널에 명령 코드 · 값 · 만료 시각 · 운전원 수락 여부 · 요청 지문을 먼저 쓰고 순번을 마지막에 씁니다(Modbus/TCP) 17. **검사 3 — PLC.** PLC는 순번이 바뀐 것을 보고 15 실행 직전에 2장의 순서대로 다시 확인한 뒤 12 출력을 바꾸고 13 7, 설비가 다음 계산에서 그 명령을 읽어 교반기가 멈춥니다 2. 압력 인터록을 푸는 일은 아예 요청할 수 있는 작업 목록에 없습니다. 인터록은 공장 안의 사람만 풉니다. 펌프 재기동은 목록에 있지만, 인터록이 잠겨 있는 동안에는 PLC가 받지 않습니다.

손으로 고치는 단계(베어링 교체 · 잠금 해제)면 접수 담당이 공장 안의 정비팀에게 작업 지시를 넘깁니다(HTTP, 정비팀 계정) 22. 정비팀 계정은 접수 담당만 갖고 있고, 호출이 실패하면 결과를 「알 수 없음」으로 남기고 다시 부르지 않습니다. 정비팀은 요청 번호마다 한 번만 일하고, 같은 번호가 또 오면 같은 결과를 돌려줍니다. 모르는 작업이면 거절합니다. 안전을 확인해 교반기나 펌프가 돌고 있거나 예비 바스켓이 많이 막혀 있으면 아무것도 건드리지 않고 「현장 안전」 사유로 거절합니다. 통과하면 설비를 격리해 잠그고(LOTO, 정비 중 기계가 갑자기 움직이지 않게 잠그는 절차로 미국 OSHA 1910.147 같은 규정이 있습니다), 작업 시간만큼 일한 뒤, 고장이 실제로 있었으면 부품 상태를 되돌리고 격리를 풉니다 104. 잠금 해제 작업이면 마지막에 정비 해제 키를 눌러 정비 모드를 끕니다 105.

이렇게 **정비팀 결과**가 태어납니다. 결과(완료 · 고장 없음), 효과가 있었는지, 소견, 걸린 시간입니다. 「구동측 베어링 외륜 박리 · 윤활 탄화 확인. 신품 교체, 유격 정상.」 엉뚱한 부품을 맡기면 정직하게 답합니다. 「베어링 분해 점검: 유격 · 소음 · 윤활 정상. 교체하지 않음. 축 정렬 점검 권고.」 다른 작업의 소견에는 스트레이너 막힘 정도, 커플링 편차, 압력계 편차, 밸브의 실제 개도 같은 단서가 들어갑니다. 요청서 한 장은 승인에서 PLC 수용과 실제 상태 확인까지 1초도 걸리지 않습니다(2026-09-30 시험). 정비 계획은 이런 요청서를 단계마다 한 장씩 보내고, 손으로 고치는 단계에는 정비팀의 작업 시간이 더해집니다.

10장 돌아오는 대답, 회복, 보고

결과는 올라온 길을 그대로 되짚어 사무실로 돌아갑니다. PLC는 응답 코드를 상태 칸에 적고 14, 엣지가 그것을 읽어 16 명령 결과 쪽지로 게시판에 붙입니다(MQTT) 18. 접수 담당은 PLC의 결과나 정비팀의 소견을 응답 쪽지로 만들어 붙이고 21, 응답은 브리지를 타고 접수처로 29, 수집기 ①이 가져가 36 장부의 request.responses 칸에 요청 번호를 열쇠로 적습니다 37.

업무 서비스(business 컨테이너)는 Kafka를 두 갈래로 읽습니다. 하나는 6장의 사건 접수(처음부터 빠짐없이), 다른 하나는 판정용(PLC 상태와 요청 응답, 최신부터, 쪽지마다 처리한 뒤 책갈피를 옮김)입니다 69. 둘을 다른 스레드와 연결로 나눈 것은, AI 승인이 사건 기록을 잠그고 결과를 기다리는 동안에도 요청 판정이 멈추지 않게 하기 위해서입니다. 판정 쪽은 ⓑ · ⓒ 응답을 정본에 요청 사건으로 적고, 짧은 주기로 진행 중인 요청을 보며 **시간 판정**을 적습니다(SQL) 70.

판정은 이렇습니다. 창구가 받은 뒤 공장 접수 응답(ⓑ)이 늦으면 「ACK 시간 초과(결과 모름)」를 적지만 그것으로 끝내지 않고, 늦게 온 응답도 기록합니다. 만료가 지나도록 ⓑ가 없으면 「만료 · 결과 미확인」, 운전원 대기에 들어간 뒤 운전원 · PLC 응답(ⓒ)이 오지 않으면 「결과 미확인」입니다. 기계를 움직이는 요청은 PLC가 받아들인 뒤, PLC 상태 칸의 값이 기대대로 바뀌었는지를 업무 서비스가 기억해 둔 PLC 상태와 비교하는 **재관측**을 합니다. 스위치는 정해진 값, 설정값은 요청 값, 정비 모드 켜기는 「켜짐」이 보이면 「확인됨」입니다. 제때 보이지 않으면 「명령과 상태가 다름」을 적고, 그 자체를 「명령 불일치」 경보로 sensor.alerts 칸에 올립니다. 그러면 그 경보는 6장의 두 갈래(AI 사건 · 알림 비서 · 운전실 화면)를 그대로 다시 지납니다. 현장 작업은 정비팀 결과가 곧 재관측입니다. 「완료」면 확인됨, 「고장 없음」이면 「확인됨 — 고장 없음」, 그 밖이거나 결과가 오지 않으면 「결과 미확인」입니다.

업무 서비스는 정비 모드가 켜지고 꺼지는 것도 감사 기록에 남깁니다(주체는 담당 번호의 운전원, 없으면 현장 패널). 감사 기록과 판정의 사본은 장부의 audit.copy · request.events 칸에도 남기는데 (71), 전달 확인 없이 보냅니다.

정비 계획 실행기는 짧은 주기로 정본의 요청 사건을 읽고(SQL) (93) 판정이 끝난 단계 다음의 요청서를 내보냅니다. 거절 · 명령 불일치 · 결과 미확인이면 그 단계는 실패입니다. 모든 단계가 끝나면 **회복**을 봅니다. 접수처 원본 사본의 최신값과 사실값으로 결정의 회복 기준을 계산해(InfluxQL) (95), 베어링이라면 진동이 충분히 내려오고 전류가 정상 범위에 든 상태가 일정 시간 유지돼야 고쳐진 것입니다. 정해진 시간 안에 유지하지 못하면 「회복 안 됨 — 원인이 남아 있을 수 있음」입니다.

마지막으로 실행기가 **작업 보고서**를 씁니다. 결과와 사유, 단계별 결과 · 요청 번호 · 정비팀 소견 · 걸린 시간, 회복 기준 센서의 정비 전후 값, 예상 대비 실적 KPI(정지 시간, 배치 폐기, 정비비, 상한 초과 운전의 위험, 품질 비용), 다음에 할 일이 들어갑니다. 결과는 넷입니다. 회복, 회복 안 됨, 원인 불일치, 단계 실패(뒤의 셋은 사건이 「미해결」로 남음). 어느 결과든 보고서는 업무 DB의 설비 정비 이력(CMMS 작업 이력)에 한 줄로 더해지고, 예상과 실적의 차이를 숨기지 않습니다. 10-06 연속 실행에서 정비 시나리오 일곱 경우(세 증상 × 두 원인, 그리고 원인을 일부러 틀리게 맡겨 본 경우)가 모두 이 끝까지 와서 회복했습니다.

AI 도우미 안에는 냉각을 켠 뒤 반응기 온도가 목표 근처에 머무는지 지켜보기만 하는 관측 담당도 있습니다. 명령은 내리지 않고, 원본 사본을 읽어 (121) 관측 기록만 남기도록(SQL) (96) 만들어져 있습니다. 하지만 이 관측은 「냉각기 켜기」 한 건 조치가 성공했을 때만 시작되는데, 9장에서 말했듯 그 대응안이 지금은 나오지 않으므로 쉬고 있습니다.

11장 같은 시간, 다른 손님들

작업 요청서를 쓰는 쪽은 AI만이 아닙니다. 생산계획 시스템(MES)도 「다음 배치는 이 온도로」 같은 지시를 보냅니다. 이 훈련 공장에서는 사무실 망 안의 시험 도구가 MES 역할로 수집기 ⑧에 주문을 넣습니다(HTTP, 인증 없음) (59). ⑧은 요청 번호를 만들고, 계획 담당을 승인자로 삼아 요청 · 「승인됨」 사건 · 감사(제안과 승인)를 SQL 한 문장으로 정본에 적은 뒤(요청자 mes-01) (60), 요청 번호를 request.approved 칸에 내고 「받음」으로 대답합니다 (61). ⑧ 자신은 내용을 검사하지 않고 DB 제약에 걸리면 기록 실패로 대답할 뿐이며, 허용 여부는 검사 1이 봅니다. 그 뒤로는 9장과 똑같은 길, 똑같은 세 겹 검사를 지납니다. 공장 입장에서는 누가 보냈든 같은 규칙입니다.

지식 담당자는 AI 화면의 **설비 지식 스튜디오**를 씁니다 (116). 자료를 올리면 언어 모델 없이 규칙대로 구조를 뽑습니다. AR-100 등록부 YAML에서는 설비 · 센서(단위 · 상하한) · 제어점 · 작업 정의를, Markdown과 텍스트에서는 문서와 절을 뽑고, PDF 같은 다른 형식은 받지 않습니다. 「AI 의미 관계 제안」은 선택 기능입니다. 질문을 주면 언어 모델이 원본 읽기와 추출 구조 읽기 두 도구로 관계 후보를 만들고(OpenAI, HTTPS) (98), 모든 원본을 읽었는지와 인용이 원문에 그대로 있는지를 검사합니다. 이 **지식 후보 생성 기록**은 AI DB(PostgreSQL)에 남고 (117), 모델이 없으면 이 기능만 꺼지고 손으로 설계하거나 후보 파일을 불러오는 길은 남습니다.

검토 화면에서는 후보마다 근거를 보고, 설비 식별자를 정리하고(소속을 추측하지 않음), 설비와 관계(센서 보유 · 설치 위치 · 절차 · 규정 · 설명 · 자산 노출)를 사유와 함께 더할 수 있습니다. **게시**는 확인 체크와 메모, 검토 내용의 지문이 일치해야 하고, 지식 지도에 한 트랜잭션으로 들어갑니다(Bolt) (97). 같은 지문은 중복으로 보고, 더 낮은 문서 버전이나 같은 설비의 다른 번호는 거절하며, 합치기만 하고 지우지 않습니다. 게시 이력, 게시된 관계 그래프(읽기 전용), 「센서는 있는데 설비와 연결되지 않은 것」 같은 누락 현황도 지식 지도에서 읽어 보여 줍니다 (133).

이렇게 게시한 설비-센서 · 설비-문서 관계는 7장의 근거 도구가 다음 사건에서 읽습니다. 다만 게시할 때 숫자 지문을 만들지 않으므로, 새로 게시한 절은 뜻으로 찾는 매뉴얼 검색에 걸리지 않습니다(그래프 채우기를 다시 돌리거나 따로 색인해야 하고, 화면에는 그 버튼이 없습니다). 증상 · 고장 모드 · 결정 같은 고장 온톨로지는 스튜디오로 만들 수 없고 그래프 채우기의 묶음으로만 들어갑니다.

AI 업무 도우미는 공개 소스 uengine-oss/ontology-studio의 백엔드와 웹 화면, process-gpt-deepagents의 일부를 가져와 고친 것입니다. 지금 켜 둔 것은 지식 스튜디오, 그래프 읽기, 파일 기능이고, 원본에 있던 채팅 세션 · 코드 실행 샌드박스 · 문서 색인과 OCR · 온톨로지 직접 편집 · 자연어 Cypher 질의 · MCP 서버는 코드만 남고 꺼져 있습니다. 그래도 기동할 때 채팅용 SQLite 저장소 준비와 샌드박스 점검, 에이전트 생성은 하므로 이것이 실패하면 서비스가 뜨지 않습니다. 같은 이미지 하나로 AI 업무 도우미 · 업무 서비스 · 그래프 채우기가 돕니다. 언어 모델은 지식 후보용과 운영 분석용 두 설정이 있고 둘 다 gpt-6-luna입니다. 이 모델은 대화형 API에서 도구 호출과 추론을 함께 쓰는 요청을 거부해서 Responses API로 직접 부르고, 중계 라이브러리는 쓰지 않습니다. API 키는 저장소 밖 파일에만 둡니다.

갈림길 순조롭지 않을 때

원인을 틀렸을 때. 실제로는 축이 틀어졌는데 베어링 교체를 맡겼다면 정비팀이 「고장 없음」으로 답합니다. 원인을 치료하는 단계에서 이 답이 오면 실행기는 남은 단계를 멈추고 「원인 불일치」로 끝냅니다(둘 다 점검하는 계획의 점검 단계라면 계속 갑니다). 이 계획이 이미 정비 모드로 잠근 뒤였다면, 실행기는 **정리 단계**로 잠금 해제 작업을 새로 요청해 정비팀이 분해한 곳을 되돌리고 격리를 풀게 하고, 설비는 멈춘 상태로 사람에게 넘깁니다. 정리 단계가 실패하면 다시 정리하지 않습니다. 사람이 다시 분석을 시작하면 AI는 방금 받은 소견을 근거로 다른 원인을 고르고, 교반기가 이미 멈춰 있으므로 「배치 끝나고 정비」 같은 대안은 규칙으로 빠집니다.

결과를 모를 때. 요청서의 대답이 오지 않으면, 도착하지 못한 것인지 실행됐는데 대답만 못 받은 것인지 알 수 없습니다. 그래서 같은 요청서를 다시 보내지 않고 그 단계를 실패로 멈춰 화면에 이유를 남깁니다. 다시 보냈다가 같은 명령이 두 번 실행되는 일을 막기 위해서입니다. 다만 이 계획이 정비 모드를 켠 뒤였다면, 어떤 실패든 잠금 해제 정리 요청 한 장은 새로 나갑니다. 업무 서비스도 그 요청을 「결과 미확인」으로 기록할 뿐 다시 보내지 않습니다.

거절되었을 때. 세 겹의 검사 어디서든 거절되면 사유가 응답(ⓐ · ⓑ · ⓒ)으로 돌아와 정본에 사건으로 남고, 실행기는 그 단계를 실패로 멈춥니다. 검사 1의 거절은 HTTP 대답으로, 검사 2와 PLC의 거절은 응답 쪽지로 같은 길을 되짚어 올라옵니다.

늦게 도착한 명령. 계측 값은 잃으면 안 되지만, 명령은 늦게 도착하면 위험합니다. 네트워크가 오래 끊겼다가 돌아온 뒤 옛날의 「교반기 정지」가 실행되면, 그사이 다시 시작한 운전이 영문도 모른 채 멈춥니다. 그래서 요청서에 일부러 짧은 만료(MQTT 5 메시지 만료와 요청서 안의 만료 시각)를 붙여, 제때 못 가면 우편함이 스스로 버리고, 공장 접수 담당과 PLC도 만료를 다시 봅니다. 끊었다 다시 이어 본 시험에서도 만료된 요청은 공장에 닿지 않았습니다. 앞에서 「전해 주면 지운다」는 걱정거리였지만, 여기서는 일부러 쓴 설계입니다.

손익 1위가 규정 위반일 때. 압력계가 실제보다 높게 가리켜 인터록이 걸린 날이라면 「인터록을 우회해 재가동」이 돈으로는 1등입니다. 그래도 규칙에 걸려 후보에서 빠지고, 카드에 「규정으로 제외」라고 적힙니다. 이 대안에는 실행 단계도 없습니다. 인터록 리셋은 운전원이 운전실이나 현장 패널에서 직접 하고, 실행기는 리셋될 때까지 기다립니다.

승인 직전에 상황이 바뀌었을 때. 카드를 만든 뒤 새 종류의 경보가 붙었거나, 스위치 · 인터록 · 정비 모드가 바뀌었거나, 문서나 설비 관계가 바뀌었거나, 기한이 지났으면 승인이 거절되고 분석은 실패로 끝납니다. 낡은 판단으로 설비를 움직이지 않기 위해서입니다.

경보가 눌렸을 때. 정비 중이거나 공장이 멈췄거나 설비를 일부러 꺼 둔 동안의 공정 경보는 알림 비서가 눌러 운전실 화면과 경보 기록에 나타나지 않습니다. 그래도 Kafka와 AI 사건 접수에는 그대로 남습니다.

연결이 끊겼을 때. 공장과 접수처 사이 브리지가 끊기면 공장은 계속 돌고, 엣지와 게시판이 쪽지를 모아 두었다가 다시 이어지면 보냅니다(감시를 켠 날에는 「브리지 끊김」 경보가 알림 비서 화면에 뜹니다). 사무실 수집기가 꺼져 있던 동안의 계측은 Kafka에 빠지고 접수처 사본에만 남으며, 그사이 승인된 요청은 다시 켜진 뒤 보내도 검사 1이 「너무 오래됨」으로 거절합니다. 발송이 대답을 제때 못 받으면 창구가 실제로는 받았더라도 거절로 기록될 수 있습니다.

설비 계산만 멈췄을 때. 가상설비의 계산이 멈추고 PLC는 살아 있으면 스캔 순번이 멈춥니다. 엣지는 새 스캔이 없으니 계측 쪽지를 더 붙이지 않고, PLC는 곧 「현장 통신 끊김」을 상태 칸에 적습니다. 엣지가 그것을 「마지막 한 장 남기기」로 붙이면 운전실 화면에 「현장 장치 통신 끊김」 알람이 뜨고, 공장 접수 담당은 그동안 오는 요청을 「PLC 사용 불가」로 거절합니다.

리니지 데이터 리니지 요약

데이터 한 종류마다 「태어나는 곳 → 바뀌는 곳 → 저장되는 곳 → 다음에 읽는 쪽」을 한 줄로 적었습니다.

센서 값: 가상설비 물리 계산 → Modbus 사물함 → PLC 입력(Modbus/TCP) → PLC의 계측 사본 칸 → 엣지가 읽어 UNS 메시지로 바꿈 → 공장 게시판(MQTT) → 운전실 화면 · 브리지 → 접수처 우편함 → (적재기) 원본 사본 InfluxDB / (수집기 ①) Kafka 계측 원본 → Flink.

PLC 상태: PLC 상태 칸 → 엣지가 상태 쪽지(retain)로 → 게시판 → 운전실 화면 · 공장 접수 담당(받을 수 있는지 판단) · 브리지 → 우편함 → ① 적재기 → 원본 사본의 PLC 상태 → AI(공정 대시보드 · 근거 · 승인 재검사 · 회복 판정) / ② 수집기 ① → Kafka plant.status → 업무 서비스(재관측 기억 · 정비 모드 감사) · 수집기 ⑤(억제 상태).

명령 결과(ACK): PLC가 응답 코드를 상태 칸에 → 엣지가 결과 쪽지로(외부 요청은 지문으로 요청 번호를 되찾음) → 게시판 → 운전실 화면 「마지막 명령 응답」 · 공장 접수 담당(PLC 응답 ⓒ로) → 브리지 → Kafka plant.status.

계측 원본 · 정리본 · 이상 점수: Kafka 계측 원본(GOOD만) → Flink ONNX 잡이 중복 제거 · 빈칸 메움 → 정리본 · 이상 점수 칸 → (감시 구성) 이력 적재기 → 결과 기록장 → Grafana.

경보: Flink 규칙 · 모델(또는 업무 서비스의 명령 불일치) → Kafka sensor.alerts → ① 수집기 ④가 꼬리표 묶음으로 → Alertmanager 묶음 · 억제 → 웹훅 → 수집기 ⑥ → 업무 DB 경보 묶음 · 사건 / ② 업무 서비스 접수 → AI DB 받은 원본 · 사건 / ③ (감시 구성) 결과 기록장.

억제 상태: PLC 상태(정비 · 정지 · 운전 꺼짐) → 수집기 ⑤ → Alertmanager의 억제 경보 → 공정 경보를 라인 전체 또는 그 설비만 누름 → 억제 시작 · 해제가 경보 사건으로.

표시 경보: 수집기 ⑥이 새 공정 경보만 한 줄로 → Kafka alerts.display → 수집기 ③ → 접수처 우편함 표시 칸(retain) → 브리지 in → 게시판 → 운전실 화면 「분석 경고(참고)」.

사건: 업무 서비스 접수가 경보를 검증 · 중복 제거 · 상관 묶기 → AI DB 사건 · 사건 이벤트 → AI 업무 도우미가 읽어 화면에 보여 주고 자동 분석 후보로 고름.

근거: AI 근거 도구가 원본 사본(사건 구간 · 최근 값)과 지식 지도(설비 관계 · 문서 · 상하한)에서 읽어 모음 → 분석 실행의 입력 · 카드의 근거 · 승인 재검사의 지식 지문.

대응안 · 정비 계획 카드: 언어 모델의 서식 맞춘 결론 + 판단 엔진의 자격 · 규칙 · 순위 · 뒤집힘 → AI DB 대응안(단계는 이때 고정) · LangGraph 체크포인트 → AI 화면 카드 → 사람.

승인: AI 화면의 결정(메모) → 검토 대기 지점에서 재개 → 승인 재검사 → 정비 계획 실행기로(또는 점검 요청 기록) → 반려면 사건이 미해결로.

작업 요청: 실행기(또는 MES 흉내 ⑧)가 업무 DB에 정본 요청 · 「승인됨」 사건 · 감사 → Kafka request.approved(번호만) → 수집기 ⑦이 정본을 꺼내 요청 창구로(HTTP) → 우편함 요청 칸(MQTT 5, 만료) → 브리지 in → 게시판 → 공장 접수 담당 → PLC 외부 요청 채널 또는 정비팀.

ⓐ **게이트웨이 응답**: 요청 창구의 HTTP 대답 → 수집기 ⑦이 업무 DB 요청 사건으로 → Kafka request.events · audit.copy 사본.

ⓑ **접수 응답 · ⓒ 운전원 · PLC · 현장 응답**: 공장 접수 담당의 응답 쪽지 → 게시판 → 브리지 → 우편함 → 수집기 ① → Kafka request.responses → 업무 서비스가 업무 DB 요청 사건으로 → 실행기가 읽음.

정비팀 결과: 가상 정비팀(격리 · 작업 · 효과 · 소견) → HTTP 대답으로 공장 접수 담당에게 → 현장 응답(ⓒ) 쪽지 → 위와 같은 길 → 업무 서비스의 재관측 판정 · 실행기의 단계 판정 · 보고서의 소견.

재관측 · 시간 판정: 업무 서비스가 요청 사건과 기억한 PLC 상태를 비교 → 업무 DB 요청 사건(확인됨 · 불일치 · 결과 미확인 · 만료 미확인) → Kafka request.events 사본 · 불일치면 경보로 다시 sensor.alerts → 실행기가 읽음.

작업 보고서 · 작업 이력: 실행기가 단계 결과 · 소견 · 회복 판정(원본 사본 최신값) · 사실값으로 작성 → AI DB 대응안 결과 · 업무 DB CMMS 작업 이력 → AI 화면 카드.

지식 후보 · 게시: 스튜디오 업로드 → 규칙 추출(+ 선택적 언어 모델 후보, AI DB에 생성 기록) → 사람 검토(지문) → 지식 지도에 병합 게시 → AI 근거 도구가 다음 사건에서 읽음(뜻 검색 지문은 만들지 않음).

등록부 생성물: 설비 등록부 → 생성기 → 엣지 흐름 · PLC 상수 · FUXA 화면 · 허용 목록과 서식 · Flink 규격 표 · 태그 → 설비 지도 · DB 등록부 표 · 태그와 작업 정의 · 온톨로지 설비 노드 → 각 부품이 기동할 때 읽음.

감시 지표: 브로커 $SYS · 엣지 · 각 부품 지표 → OT 에이전트(remote write) · DMZ Prometheus → IT Prometheus(federate) → 규칙 경보는 Alertmanager 기본 경로(보낼 곳 없음) · Grafana 인프라 대시보드.

한계 알려진 한계 (2026-10-07 코드 기준)

인증과 노출. AI 화면과 AI 업무 도우미 API에는 로그인이 없고 어느 출처의 요청이든 받도록(CORS 전체 허용) 되어 있어, 같은 PC나 사무실 망의 누구든 승인 · 게시 · 파일 삭제를 부를 수 있습니다(화면에도 「검토자 신원 인증 아직 없음」이라고 적혀 있습니다). 요청서의 「승인자」 칸에는 화면에서 실제로 승인한 사람이 아니라 설정에 정해 둔 운영자가 들어갑니다(실제 승인 메모는 AI 쪽 기록에 남음). Kafka의 PC 쪽 포트는 인증이 없고, cAdvisor는 특권으로 실행되며 호스트를 들여다보고, Grafana는 로그인 없이 볼 수 있습니다.

잃거나 놓칠 수 있는 것. AI 사건 접수 중 DB 오류가 나면 그 경보는 영구히 빠질 수 있고, 업무 서비스의 판정 쪽도 처리 중 오류가 난 쪽지를 되감지 않습니다. 업무 서비스가 Kafka에 남기는 사본(감사 · 요청 사건)은 전달 확인이 없습니다. 판정은 최근에 만든 요청만 봅니다. 엣지 디스크 큐와 PLC의 정비 상태는 주기적으로 저장되므로 갑자기 꺼지면 그 사이를 잃을 수 있고, 접수처 적재기와 사무실 수집기의 묶음 버퍼는 메모리에 있습니다. Flink의 경보 출력은 「최소 한 번」이라 복구 뒤 중복이 생길 수 있습니다.

기억이 사라지는 것. 접수처 요청 창구의 중복 기억과 공장 접수 담당의 중복 · 대기 기억은 메모리에만 있어, 다시 켜지면 사라지고 대기 중이던 요청은 「결과 미확인」으로 끝납니다. PLC의 중복 기억도 전원 유지 영역이 아니라서 PLC가 다시 켜지면 만료로만 거릅니다. PLC가 다시 켜지면 구동 명령의 기본값이 「켜짐」이라 정비 중이었어도 교반기 명령이 되살아납니다(정비팀이 격리한 동안만 물리적으로 멈춰 있습니다). 운전원 채널에는 만료 · 중복 · 시각 검사가 없고 엣지의 짧은 나이 검사만 있습니다. 가상설비를 다시 켜면 상태와 패널 버튼 카운터가 처음으로 돌아가는데, PLC는 카운터가 바뀐 것을 버튼이 눌린 것으로 보므로 엉뚱한 패널 동작으로 읽힐 수 있습니다.

판단과 경보의 빈틈. 베어링 긴급 조달 시간은 사실값에 있지만 계산에 쓰이지 않아, 예비 베어링이 없으면 「주문하고 기다리는」 안 없이 베어링 대안이 모두 빠집니다. 한 건짜리 대응안(교반기 정지 · 냉각기 켜기)과 냉각 뒤 온도 관측은 코드에 있지만 지금 AI 출력 종류로는 닿지 않습니다. 고착된 히터는 비상정지 중에도 가열합니다(고장의 성격으로 그렇게 만들었습니다). Alertmanager의 첫째 억제 규칙(「서비스가 죽으면 파생 경보를 누른다」)은 원인 경보에 계층 꼬리표가 없어 사실상 동작하지 않을 것으로 보입니다. 인프라 경보는 보낼 곳이 없어 화면에서만 보입니다. Grafana는 냉각수 쪽 센서를 그리지 않고, 연결해 둔 접수처 원천을 쓰는 패널도 없습니다. 품질 UNCERTAIN은 정의만 있고, 변경 관리 표는 쓰는 곳이 없습니다.

운영할 때 주의할 점. 등록부를 고친 뒤의 「다시 만들기」는 PLC · 엣지 · 요청 창구를 다시 빌드하고 화면 넣기를 다시 돌리지만, 수집기 · 업무 서비스 · AI 업무 도우미는 다시 켜야 새 설정을 읽고, Flink 규격 표는 잡을 다시 제출해야 하며, 업무 DB의 등록부 표는 빈 볼륨 첫 기동 때만 만들어지고, 지식 지도는 그래프 채우기를 다시 돌려야 바뀝니다. 「모델 다시 학습」은 새 모델을 만든 뒤 잡 제출기를 돌리지만, 잡이 이미 모두 돌고 있으면 제출기가 건너뛰므로 돌던 ONNX 잡은 옛 모델을 계속 씁니다(잡을 내리고 다시 제출해야 함). 배속 값은 가상설비 · PLC 상수 · AI 쪽에 따로 적혀 있어 함께 바꿔야 합니다. 엣지 편집기에서 고친 흐름은 다시 켜면 사라지고, 지식 스튜디오로 게시한 절은 뜻 검색에 바로 걸리지 않습니다. 컨테이너마다 메모리 상한은 걸지 않았습니다.

저장소 밖 사실의 출처

이 시스템에 관한 내용은 2026-10-07 코드 · 설정과 날짜가 적힌 시험 기록이 근거입니다. 아래는 이야기에 쓴 일반 사실(표준 · 프로토콜)을 확인한 곳입니다.

- Modbus: 1979년 Modicon 발표, 인증 · 암호화 · 무결성 없음 — https://www2.cs.siu.edu/~aydeger/classes/CS%20531/8.ICS-Protocols.pdf
- ISA-95 설비 계층: 엔터프라이즈 · 사이트 · 구역 · 라인 — https://documentation.iconics.com/v11/Content/Assets/ISA95-and-ISA88-hierarchical-structures.htm
- Mosquitto autosave_interval: 메모리 내용을 디스크에 저장하는 간격 — https://manpages.org/mosquittoconf/5
- MQTT 5 메시지 만료: 기간이 지나면 브로커가 버림 — https://www.vernemq.com/blog/2019/12/27/mqttv5-message-expiry.html
- Kafka 소비자 오프셋: 그룹마다 읽은 위치를 기록해 재시작 뒤 이어 읽음 — https://docs.confluent.io/kafka/design/consumer-design.html
- ANSI/ISA-18.2: 공정 산업 경보 관리, 경보 홍수 — https://www.controleng.com/articles/implementing-alarm-management-per-the-ansi-isa-18-2-standard/
- ISA-18.2 경보 상태: 보류(Shelved) · 설계상 억제(Suppressed by Design) · 사용 중지(Out of Service) — https://documentation.iconics.com/v10.97.3/Content/Alarming/Alarm%20Server/Alarm%20References/alarm-state-transition-diagram.htm
- IEC 61511: 안전 계장 시스템과 일반 제어의 분리 · 독립 — https://www.emersonautomationexperts.com/2009/safety/views_on_safety
- IEC 62443: 구역(zone)과 관문(conduit), IT와 OT 사이 DMZ — https://wz-it.com/en/knowledge/remote-access/ot-it-segmentation-dmz-remote-access/
- OSHA 29 CFR 1910.147: 정비 중 위험 에너지 차단(LOTO) — https://osha.gov/control-hazardous-energy/standards
- 균형 성과표(BSC)의 관점: 재무 · 고객 · 내부 프로세스 · 학습과 성장 — https://leandatapoint.com/resources/balanced-scorecard-four-perspectives
- OpenPLC Runtime v4: IEC 61131-3(ST) 실행, Modbus 슬레이브 · 원격 장치(마스터) — https://github.com/Autonomy-Logic/openplc-runtime/releases
- node-red-contrib-modbus: Modbus TCP 읽기 · 쓰기 — https://flows.nodered.org/node/node-red-contrib-modbus
- Mosquitto 설정: 익명 금지 · 비밀번호 · ACL · MQTT 5 브리지 · 영속 · 대기열 한도 — https://mosquitto.org/man/mosquitto-conf-5.html
- MQTT 5.0 명세(메시지 만료 등) — https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html
- FUXA: 웹 SCADA/HMI, MQTT 지원 — https://github.com/frangoteam/FUXA/releases
- Caddy basic_auth — https://caddyserver.com/docs/caddyfile/directives/basic_auth
- Bento(WarpStream): 스트림 처리기 — https://github.com/warpstreamlabs/bento/releases
- InfluxDB 2.x v1 호환 InfluxQL 조회 — https://docs.influxdata.com/influxdb/v2/api-guide/influxdb-1x/

- Apache Kafka 4.x(KRaft 전용) — https://kafka.apache.org/community/downloads/
- Flink: SQL · MATCH_RECOGNIZE(CEP) · 체크포인트 · ZooKeeper HA — https://flink.apache.org/downloads/
- Prometheus agent 모드 · remote write — https://prometheus.io/docs/prometheus/latest/prometheus_agent/
- Alertmanager 묶음 · 억제 · 웹훅 — https://prometheus.io/docs/alerting/latest/configuration/
- Grafana InfluxDB(Flux) 데이터 원천 — https://grafana.com/docs/grafana/latest/datasources/influxdb/
- PostgreSQL 버전 정책 — https://www.postgresql.org/support/versioning/
- Neo4j 5 LTS · 벡터 인덱스 — https://neo4j.com/blog/developer/neo4j-v5-lts-evolution/
- LangGraph 체크포인터 · interrupt(사람 승인 대기) — https://reference.langchain.com/python/langgraph/checkpoints/
- OpenAI 임베딩 text-embedding-3-small — https://developers.openai.com/api/docs/guides/embeddings
- iptables DNAT(nat PREROUTING) — https://man7.org/linux/man-pages/man8/iptables-extensions.8.html